



OpenClaw

Manual Oficial de Instalación, Configuración y Administración

Una guía completa e independiente de la plataforma:
Windows, Linux, macOS, contenedores o servidores remotos. Desde los fundamentos de la terminal hasta la administración avanzada del sistema.

Compilado y editado por Fernando García del Castillo López
Basado en la documentación oficial (docs.openclaw.ai) · OpenClaw es software libre con licencia MIT

Licencia y atribución

Sobre este documento

Este manual ha sido compilado y editado por **Fernando García del Castillo López**, a partir de la documentación oficial de OpenClaw y de la experiencia práctica de instalación y configuración de la plataforma. El contenido está organizado y redactado de forma original para servir como guía de aprendizaje y referencia, independiente del método de instalación o la plataforma elegidos.

Sobre OpenClaw

OpenClaw es un proyecto de software de código abierto, publicado bajo **licencia MIT**. Esta es una de las licencias de software libre más permisivas que existen: permite usar, copiar, modificar, fusionar, publicar, distribuir, sublicenciar y vender copias del software, tanto del proyecto original como de obras derivadas, prácticamente sin restricciones, con la única condición habitual de conservar el aviso de copyright y la propia licencia en las copias del software.

Licencia de este manual

Siguiendo ese mismo espíritu de software y conocimiento libres, este manual se distribuye con **libre acceso y distribución**: puedes copiarlo, compartirlo, imprimirlo y modificarlo libremente, siempre que mantengas la atribución a su autor y, si lo redistribuyes modificado, indiques los cambios realizados. No se reclama ninguna restricción adicional sobre el contenido más allá de mantener este crédito.

Nota de transparencia: el autor de este manual no es el autor del software OpenClaw ni está afiliado a su desarrollo. El crédito de autoría aquí indicado corresponde exclusivamente a la redacción, organización y compilación de este documento como material didáctico.

Índice general

Parte I. Fundamentos: el ordenador por dentro

1. Qué es este manual y cómo usarlo
2. La terminal: hablar con el ordenador escribiendo
3. PowerShell, CMD y Bash: los tres intérpretes
4. Comandos esenciales explicados uno a uno

Parte II. Virtualización y contenedores

5. Qué es la virtualización y por qué existe
6. WSL2: Linux dentro de Windows
7. Docker desde cero: imágenes, contenedores y volúmenes
8. Comandos de Docker explicados

Parte III. OpenClaw

9. Qué es OpenClaw y para qué sirve
10. Arquitectura: gateway, agente, canales y modelos

Parte IV. Elige tu método de instalación

11. Mapa de decisión: qué método y dónde
12. Método A — Node.js nativo (cualquier sistema operativo)
13. Método B — Docker con imagen preconstruida
14. Método C — Docker con imagen oficial y Compose
15. Método D — Servidor remoto / VPS
16. El onboarding pregunta a pregunta (todos los métodos)

Parte V. Modelos de IA

17. Qué es un modelo de lenguaje y cómo se elige
18. Conectar un proveedor de modelos
19. Tokens, contexto y velocidad

Parte VI. Configuración y sistema de archivos

20. Dónde vive .openclaw según el método elegido
21. openclaw.json: la configuración técnica
22. El workspace: la personalidad del agente
23. Búsqueda web y herramientas

Parte VII. Operación y resolución de problemas

24. Operar el gateway en el día a día (según el método)
25. Leer logs y diagnosticar
26. Catálogo de problemas reales y soluciones
27. Seguridad

Parte VIII. Temas avanzados

- 28. Multi-agente
- 29. Canales: hablar desde el móvil
- 30. Copias de seguridad, portabilidad y migración entre métodos

Parte IX. Git y GitHub

- 31. Qué es el control de versiones
- 32. Git y GitHub paso a paso
- 33. Documentar el proyecto para un portfolio

Parte X. Práctica: Linux y uso real del asistente

- 34. Linux desde cero: comandos para sobrevivir
- 35. Usar el asistente para estudiar o trabajar
- 36. Resumen del flujo completo

Parte XI. Fundamentos complementarios

- 37. Redes desde cero: IP, puertos y localhost
- 38. JSON desde cero: para no romper la configuración
- 39. El panel de control por dentro
- 40. Preguntas frecuentes

Parte XII. Para profundizar

- 41. Cómo funciona la IA por dentro (sin matemáticas)
- 42. Automatización: hooks y tareas programadas

Parte XIII. La consola de administración

- 43. Mapa general del panel de administración
- 44. Ajustes: Comunicaciones
- 45. Ajustes: Apariencia
- 46. Ajustes: Automatización
- 47. Ajustes: MCP y ACP — extensibilidad avanzada
- 48. Ajustes: Infraestructura
- 49. Ajustes: IA y Agentes
- 50. Depuración y Registros
- 51. Grupo Control: monitorización del gateway
- 52. Grupo Agente: identidad y capacidades

Apéndices

- A. Chuleta de comandos (PowerShell, Bash, Docker, OpenClaw)
- B. Glosario completo
- C. Recursos y enlaces

PARTE I

Fundamentos: el ordenador por dentro

Antes de tocar OpenClaw, conviene entender el terreno: qué es una terminal, qué diferencia hay entre PowerShell y Bash, y qué hacen realmente los comandos que vas a teclear.

1. Qué es este manual y cómo usarlo

Este documento parte de cero absoluto. No da por sabido nada: ni qué es una terminal, ni qué es Docker, ni qué es una API. Si ya conoces algún tema, puedes saltar ese capítulo; si no, está pensado para leerse en orden.

El objetivo es que, al terminar, entiendas no solo *cómo* se monta un asistente de IA autoalojado como OpenClaw, sino **por qué** cada paso es necesario. La diferencia entre seguir un tutorial a ciegas y comprender lo que haces es enorme: cuando algo falla —y siempre falla algo—, solo quien entiende el porqué sabe arreglarlo.

1.1. Un manual, varios caminos

OpenClaw se puede instalar de formas muy distintas: en tu propio ordenador con Windows, Linux o macOS, dentro de un contenedor Docker, o en un servidor remoto al que accedes por internet. Este manual no asume ninguna de estas opciones como "la normal" — las trata todas como caminos válidos, y en cada capítulo de instalación y configuración encontrarás las variantes una junto a otra, señaladas con pestañas de método, para que sigas la que corresponda a tu caso sin tener que adivinar qué parte del texto te aplica.

A lo largo del manual verás bloques marcados como **[Windows]**, **[Linux/macOS]**, **[Docker]** o **[Servidor remoto]** cuando un paso varía según dónde instales. Si no ves ninguna marca, el contenido es igual en todos los casos.

1.2. Cómo está organizado

El manual se divide en trece partes, de lo más general a lo más específico:

- **Parte I — Fundamentos:** la terminal, los intérpretes de comandos y los comandos básicos.
- **Parte II — Virtualización:** qué es WSL2 y qué es Docker, explicados desde los cimientos.
- **Parte III — OpenClaw:** qué es y cómo está hecho por dentro.
- **Parte IV — Elige tu método:** las distintas formas de instalar, con sus pasos completos.
- **Parte V — Modelos:** cómo funciona la IA que da las respuestas y cómo conectar un proveedor.
- **Parte VI — Configuración:** los archivos, la personalidad del agente y las herramientas.
- **Parte VII — Operación:** el día a día, leer logs y resolver problemas.
- **Parte VIII — Avanzado:** multi-agente, canales de mensajería, copias de seguridad.
- **Parte IX — Git y GitHub:** cómo documentar y publicar un proyecto basado en esto.
- **Partes X-XII — Práctica y profundización:** Linux, redes, JSON, automatización.
- **Parte XIII — La consola de administración:** recorrido completo del panel web.

1.3. Convenciones tipográficas

Para que sea fácil de leer, el manual usa estos elementos:

- Los `comandos y nombres de archivo` aparecen en este formato monoespaciado.
- Los bloques de código (lo que escribes en la terminal) van en cajas oscuras.
- Las teclas se muestran así: `Ctrl` + `C`.

Nota: los recuadros azules dan información complementaria útil.

¿Por qué? los recuadros amarillos explican la razón de fondo de algo. Son la clave para *entender* en vez de solo *copiar*.

Cuidado: los recuadros rojos avisan de errores frecuentes o riesgos de seguridad.

Consejo: los recuadros verdes dan trucos prácticos.

1.4. Qué vas a construir

Al final del manual tendrás montado un **asistente de inteligencia artificial personal** que corre en el entorno que tú elijas: tu propio ordenador, un contenedor aislado, o un servidor remoto. Será privado (tus conversaciones no salen de donde tú decidas, salvo que lo configures), personalizable (puedes darle nombre, personalidad y memoria) y accesible desde el navegador o incluso desde el móvil.

2. La terminal: hablar con el ordenador escribiendo

Antes de las interfaces gráficas, los ordenadores se manejaban escribiendo órdenes de texto. Esa forma de comunicarse no ha desaparecido: sigue siendo la herramienta más potente para tareas técnicas, y es donde vivirás buena parte de este manual, sea cual sea tu sistema.

2.1. Interfaz gráfica vs línea de comandos

Hay dos formas de darle órdenes a un ordenador:

- **Interfaz gráfica (GUI):** lo que usas a diario. Haces clic en iconos, arrastras ventanas, pulsas botones. Es intuitiva, pero limitada: solo puedes hacer lo que los botones permiten.
- **Línea de comandos (CLI):** escribes órdenes de texto y el ordenador las ejecuta. Es menos intuitiva al principio, pero infinitamente más potente y precisa.

¿Por qué usar la terminal si hay ventanas? Porque muchas herramientas técnicas —Docker, Git, OpenClaw— se controlan principalmente por texto. Un solo comando puede hacer lo que en una interfaz gráfica requeriría veinte clics, y además queda escrito, así que puedes repetirlo, guardarlo y compartirlo. Para un desarrollador, la terminal no es opcional: es el día a día.

2.2. Qué es exactamente una "terminal"

Hoy, una terminal es simplemente una **ventana donde escribes comandos de texto y ves las respuestas**. También se la llama "consola" o "línea de comandos". Esto es igual en Windows, Linux y macOS; lo que cambia es qué programa la gestiona (lo verás en el próximo capítulo) y qué aspecto tiene.

Cuando abres una terminal, ves algo parecido a esto:

```
usuario@maquina:~$ _
```

Eso de la izquierda es el **prompt** (el "indicador"): el ordenador te dice "estoy listo, escribe tu orden". Cuando escribes un comando y pulsas `Enter`, el ordenador lo ejecuta y muestra el resultado.

2.3. Anatomía de un comando

Casi todos los comandos siguen la misma estructura:

```
programa  subcomando  opciones  argumentos
```


Por ejemplo, en este comando que usarás más adelante:

```
docker run -d --name openclaw imagen:latest
```

- `docker` → el **programa** que ejecutas.
- `run` → el **subcomando** (qué quieres que haga docker: "ejecutar").
- `-d` y `--name openclaw` → las **opciones** (modificadores que ajustan el comportamiento).
- `imagen:latest` → el **argumento** (sobre qué actúa el comando).

Nota sobre las opciones: las que empiezan con un guion (`-d`) suelen ser abreviaturas de una letra; las que empiezan con dos guiones (`--name`) son la versión larga y legible. Muchas veces `-d` y `--detach` significan lo mismo.

2.4. Errores: son normales, no los temas

Vas a equivocarte al escribir comandos, y el ordenador te lo dirá con mensajes de error. Eso es **parte del proceso**, no un fracaso. Aprender a leerlos es una de las habilidades más valiosas que desarrollarás. Más adelante hay un capítulo entero dedicado a interpretar errores y logs.

Consejo: si un comando no funciona, lee el mensaje completo con calma. La mayoría de errores dicen exactamente qué falta o qué está mal. Y si no lo entiendes, copiarlo en un buscador (o preguntárselo a tu propio asistente de IA, una vez montado) casi siempre da la respuesta.

3. PowerShell, CMD y Bash: los tres intérpretes

No existe "una" terminal: existen varios programas que interpretan los comandos que escribes. En este proyecto te cruzarás con tres. Entender en qué se diferencian evita mucha confusión, especialmente cuando un comando funciona en uno pero no en otro.

3.1. El intérprete (shell)

Cuando escribes un comando, hay un programa que lo lee, lo entiende y lo ejecuta: el **intérprete de comandos**, en inglés *shell*. La terminal es la ventana; el shell es el "cerebro" que hay detrás interpretando lo que escribes. Distintos sistemas usan distintos shells, y cada uno tiene su propia sintaxis (su forma de escribir las órdenes).

3.2. Los tres que vas a encontrar

Shell	Dónde vive	Para qué lo usarás aquí
PowerShell	Windows (moderno)	Lanzar comandos de Docker desde Windows. Es tu base de operaciones principal.
CMD	Windows (clásico)	Casi nada. Es el viejo "símbolo del sistema". Lo menciono para que lo reconozcas.
Bash	Linux / WSL / dentro de los contenedores	La documentación de la mayoría de proyectos usa Bash. También lo usarás dentro de la VM de Linux y dentro del contenedor.

3.3. Diferencias prácticas que te afectarán

La diferencia más visible entre PowerShell y Bash, y la que más quebraderos de cabeza da, es cómo se parten los comandos largos en varias líneas. Cuando un comando es muy largo, se suele escribir en varias líneas usando un carácter de continuación al final de cada una. Y ese carácter **es distinto en cada shell**:

Concepto	Bash (Linux/Mac)	PowerShell (Windows)
Continuar en la línea siguiente	barra invertida <code>\</code>	acento grave <code>`</code>
Variable de entorno	<code>\$HOME</code>	<code>\$env:HOME</code> o <code>\${HOME}</code>
Carpeta del usuario	<code>~</code> o <code>\$HOME</code>	<code>\$env:USERPROFILE</code>
Separador de rutas	barra normal <code>/</code>	barra invertida <code>\</code>

¿Por qué importa esto? Porque la documentación de OpenClaw (y de casi todo el software libre) está escrita para **Bash**, usando `\` y `$HOME`. Si copias esos comandos tal cual en PowerShell, fallarán. Tienes que "traducirlos": cambiar `\` por ``` y `$HOME` por `${HOME}`. Saber esto te ahorra horas de "no entiendo por qué no funciona".

Mira el mismo comando en los dos shells. En **Bash**:

```
docker run -d \
  --name openclaw \
  -v $HOME/.openclaw:/home/node/.openclaw \
  imagen:latest
```

El mismo comando en **PowerShell**:

```
docker run -d `
  --name openclaw `
  -v ${HOME}\.openclaw:/home/node/.openclaw `
  imagen:latest
```

Fíjate: cambian solo el carácter del final de línea (`\` → ```) y la variable (`$HOME` → `${HOME}`). El resto es idéntico.

Consejo: si un comando largo te da problemas en PowerShell, una solución infalible es escribirlo **todo en una sola línea**, sin caracteres de continuación. Es más feo de leer, pero elimina el problema de los `\` y ``` de golpe.

3.4. Cómo abrir cada terminal

- **PowerShell:** pulsa `Win`, escribe "PowerShell" y ábrelo. O clic derecho en el menú de inicio → "Terminal" / "Windows PowerShell".
- **Bash (en WSL):** si tienes Ubuntu instalado, búscalo en el menú de inicio. O escribe `wsl` en PowerShell.
- **Bash (dentro de un contenedor):** con `docker exec -it <contenedor> bash` (lo verás más adelante).

4. Comandos esenciales explicados uno a uno

Aquí tienes los comandos de terminal que de verdad vas a necesitar, explicados con calma y con ejemplos. No es una lista exhaustiva: es lo justo y necesario para no perderte.

4.1. Moverse por las carpetas

El sistema de archivos es como un árbol de carpetas dentro de carpetas. En la terminal siempre estás "situado" en una carpeta concreta (la carpeta actual o "directorio de trabajo"). Estos comandos te permiten moverte y ver dónde estás:

Comando (Bash)	Comando (PowerShell)	Qué hace
<code>pwd</code>	<code>pwd</code>	Muestra en qué carpeta estás ahora ("print working directory").
<code>ls</code>	<code>ls</code> o <code>dir</code>	Lista los archivos y carpetas del sitio donde estás.
<code>cd carpeta</code>	<code>cd carpeta</code>	Entra en una carpeta ("change directory").
<code>cd ..</code>	<code>cd ..</code>	Sube a la carpeta de arriba (la "carpeta madre").
<code>cd ~</code>	<code>cd ~</code>	Va a tu carpeta personal de usuario.

Ejemplo de uso real, paso a paso:

```
PS C:\Users\fernando> pwd
C:\Users\fernando
PS C:\Users\fernando> cd Documentos
PS C:\Users\fernando\Documentos> ls
# aquí saldría la lista de archivos de Documentos
```

4.2. Crear, copiar y borrar

Bash	PowerShell	Qué hace
<code>mkdir nombre</code>	<code>mkdir nombre</code>	Crea una carpeta nueva.
<code>cp origen destino</code>	<code>copy origen destino</code>	Copia un archivo.
<code>mv origen destino</code>	<code>move origen destino</code>	Mueve o renombra un archivo.
<code>rm archivo</code>	<code>del archivo</code>	Borra un archivo. Cuidado: no va a la papelera.
<code>cat archivo</code>	<code>type archivo</code> o <code>cat</code>	Muestra el contenido de un archivo de texto.

Cuidado con `rm` y `del` : en la terminal, borrar es **definitivo**. No hay papelera de reciclaje. Antes de borrar algo, asegúrate de que es lo que quieres. Esta es una de las razones por las que conviene ir con cuidado y leer dos veces antes de pulsar Enter.

4.3. Variables de entorno

Una **variable de entorno** es un valor con nombre que el sistema guarda y que los programas pueden leer. Es como una etiqueta con un dato dentro. Las usarás para guardar tu clave de la API de NVIDIA sin tener que escribirla cada vez.

Crear una variable en **PowerShell**:

```
$env:NVIDIA_API_KEY = "nvapi-tu-clave-aqui"
```

Crear una variable en **Bash**:

```
export NVIDIA_API_KEY="nvapi-tu-clave-aqui"
```

Leer (mostrar) una variable:

```
# PowerShell
echo $env:NVIDIA_API_KEY
# Bash
echo $NVIDIA_API_KEY
```

¿Por qué usar variables de entorno para las claves? Por dos razones. Primero, comodidad: la defines una vez y los programas la leen solos. Segundo, y más importante, **seguridad**: así la clave no queda escrita "a la vista" en cada comando ni en archivos que podrías subir por error a internet. Es la forma profesional de manejar secretos.

Cuidado: una variable creada con `$env:` o `export` solo vive en **esa ventana de terminal**. Si la cierras y abres otra, desaparece. Para que sea permanente hay que guardarla en la configuración del shell (en Bash, en el archivo `~/.bashrc`). Más adelante se explica cómo.

4.4. Atajos de teclado imprescindibles

Atajo	Qué hace
<code>↑</code> / <code>↓</code>	Recorre los comandos que escribiste antes (historial). Ahorra muchísimo tecleo.
<code>Tab</code>	Autocompleta nombres de archivos y carpetas. Escribe las primeras letras y pulsa Tab.
<code>Ctrl</code> + <code>C</code>	Cancela el comando que se está ejecutando ahora. Tu botón de "para".
<code>Ctrl</code> + <code>L</code>	Limpia la pantalla (también <code>clear</code> en Bash o <code>cls</code> en PowerShell).

Consejo: la tecla `Tab` es tu mejor amiga. Además de ahorrar tiempo, evita errores de escritura: si autocompleta, es que el archivo existe; si no autocompleta, es que has escrito mal el nombre o no estás en la carpeta correcta.

PARTE II

Virtualización y contenedores

El corazón técnico del proyecto. Qué es Docker, por qué existe, y cómo permite ejecutar OpenClaw en una "caja" aislada sin ensuciar tu sistema.

5. Qué es la virtualización y por qué existe

Para entender Docker, primero hay que entender un problema viejo de la informática: cómo hacer que un programa funcione igual en cualquier ordenador, sin importar qué tenga instalado.

5.1. El problema de "en mi máquina funciona"

Imagina que escribes un programa en tu ordenador y funciona perfecto. Se lo pasas a un compañero y... no funciona. ¿Por qué? Porque su ordenador tiene una versión distinta de algo, le falta una librería, o tiene una configuración diferente. Este problema es tan clásico que tiene nombre propio: "en mi máquina funciona". Durante décadas fue una pesadilla.

5.2. La idea de empaquetar el entorno completo

La solución conceptual es elegante: en lugar de pasar solo el programa, pasas el programa **junto con todo lo que necesita para funcionar** — el sistema, las librerías, la configuración, todo metido en un paquete. Así, ese paquete funciona igual en cualquier sitio, porque lleva su entorno consigo. Eso es, en esencia, lo que hace la virtualización moderna.

5.3. Máquinas virtuales vs contenedores

Hay dos formas de lograr ese aislamiento, y conviene distinguirlas porque en este proyecto usarás ambas (una para Linux de prácticas, otra para OpenClaw):

	Máquina virtual (VM)	Contenedor (Docker)
Qué simula	Un ordenador entero, con su propio sistema operativo completo.	Solo el entorno de una aplicación, compartiendo el núcleo del sistema anfitrión.
Peso	Pesado (gigas). Arranca en minutos.	Ligero (megas). Arranca en segundos.
Ejemplo en este proyecto	La VM de Debian en VirtualBox donde practicas Linux.	El contenedor donde corre OpenClaw.

¿Por qué OpenClaw va en un contenedor y no en una VM? Porque OpenClaw es una sola aplicación, no necesita un ordenador entero simulado. Un contenedor le da el aislamiento que necesita (no ensucia tu Windows) pero sin el peso de una máquina virtual completa. Es la herramienta adecuada para el trabajo: ligera, rápida y desechable.

6. WSL2: Linux dentro de Windows

Docker, por su naturaleza, está pensado para Linux. En Windows necesita una capa que le dé un Linux por debajo. Esa capa es WSL2.

6.1. Qué es WSL2

WSL son las siglas de *Windows Subsystem for Linux* (Subsistema de Windows para Linux). Es una tecnología de Microsoft que permite **ejecutar Linux dentro de Windows**, de forma integrada y eficiente. El "2" indica que es la segunda versión, mucho más rápida y completa que la primera.

En la práctica, WSL2 te da un Linux real funcionando dentro de tu Windows, sin necesidad de reiniciar ni de instalar una máquina virtual pesada. Docker Desktop lo usa por debajo para tener el Linux que necesita.

6.2. WSL2 y Docker: cómo encajan

Cuando instalas Docker Desktop en Windows, este se apoya en WSL2 para funcionar. Tú no tienes que hacer casi nada: Docker Desktop configura WSL2 por su cuenta. El resultado es que puedes ejecutar contenedores Linux desde tu Windows con normalidad.

Nota: es posible que en tu sistema veas una distro de WSL llamada `docker-desktop`. Esa es **interna de Docker**: la usa Docker para sí mismo. No es un Linux donde trabajes ni guardes archivos. Si quieres un Linux propio para teclear comandos Bash, instala Ubuntu (con `wsl --install -d Ubuntu`), aunque para este proyecto, como verás, ni siquiera hace falta: PowerShell basta.

6.3. Comandos básicos de WSL

Comando (en PowerShell)	Qué hace
<code>wsl --list --verbose</code>	Lista las distribuciones de Linux instaladas y su estado.
<code>wsl --install -d Ubuntu</code>	Instala Ubuntu como distro de WSL.
<code>wsl</code>	Entra en tu distro de Linux por defecto.

¿Por qué no necesitas Ubuntu para este proyecto? Porque los comandos de Docker funcionan directamente desde PowerShell. WSL2 trabaja por debajo (Docker lo usa), pero tú le das las órdenes desde PowerShell sin tener que entrar a ningún Linux. Ubuntu solo haría falta si quisieras una terminal Bash propia, cosa que en este flujo no es necesaria.

7. Docker desde cero: imágenes, contenedores y volúmenes

Este es el capítulo más importante de la Parte II. Si entiendes bien estos tres conceptos — imagen, contenedor y volumen— entenderás todo lo que viene después.

7.1. La analogía de la cocina

Docker tiene tres conceptos centrales que se entienden bien con una analogía culinaria:

- La **imagen** es como una *receta congelada*: contiene las instrucciones y los ingredientes para preparar un plato. Es estática, no cambia.
- El **contenedor** es el *plato ya cocinado y servido* a partir de esa receta. Es la versión "viva" y en funcionamiento. De una misma receta (imagen) puedes cocinar muchos platos (contenedores).
- El **volumen** es como un *táper* donde guardas comida para que no se pierda cuando tiras el plato. Permite que los datos sobrevivan aunque borres el contenedor.

7.2. La imagen

Una **imagen Docker** es una plantilla de solo lectura que contiene todo lo necesario para ejecutar una aplicación: el sistema base, las librerías, el propio programa y su configuración por defecto. Las imágenes se guardan en almacenes públicos llamados *registros* (el más conocido es Docker Hub; otro es GitHub Container Registry, ghcr.io).

Para obtener una imagen, se usa `docker pull`:

```
docker pull ghcr.io/usuario/openclaw-docker:latest
```

Eso descarga la imagen a tu ordenador. Aún no la has ejecutado: solo la tienes guardada, lista para usar. El `:latest` del final es la *etiqueta* (tag), que indica la versión ("latest" = la más reciente).

¿Por qué "pull" y no "git clone"? Aunque ghcr.io es de GitHub, una imagen Docker no es código fuente que se clona con Git: es un paquete binario ya construido. Por eso se baja con `docker pull` (la herramienta de Docker) y no con `git clone` (la herramienta de Git). Son dos cosas distintas que a veces se confunden por estar ambas relacionadas con GitHub.

7.3. El contenedor

Un **contenedor** es una imagen en ejecución. Cuando ejecutas una imagen, Docker crea un contenedor: una instancia viva, aislada del resto del sistema, donde la aplicación corre. Se crea con `docker run`:

```
docker run -d --name openclaw imagen:latest
```

Puntos clave de los contenedores:

- Están **aislados**: lo que pasa dentro no afecta a tu sistema, y viceversa (salvo lo que tú conectes explícitamente).
- Son **desechables**: puedes crearlos y destruirlos a voluntad. Esto es una virtud, no un defecto.
- Por defecto, **no guardan datos de forma permanente**: si borras el contenedor, lo que había dentro se va con él. Para evitarlo existen los volúmenes.

7.4. El volumen: la pieza que salva tus datos

Aquí está el concepto que más confunde y más importa. Como los contenedores son desechables, si guardaras los datos importantes *dentro* del contenedor, los perderías cada vez que lo recrearas. La solución es el **volumen** (o "bind mount"): conectar una carpeta de tu ordenador (el "anfitrión" o *host*) con una carpeta de dentro del contenedor.

Se hace con la opción `-v` al crear el contenedor:

```
-v C:\Users\fernando\.openclaw:/home/node/.openclaw
```

Esto significa: "la carpeta `.openclaw` de mi Windows queda conectada a la carpeta `/home/node/.openclaw` de dentro del contenedor". Todo lo que la aplicación guarde ahí dentro, aparece en tu Windows, y sobrevive aunque borres el contenedor.

¿Por qué es tan importante el volumen? Porque separa dos cosas con ciclos de vida distintos: el **software** (el contenedor, que es efímero y reemplazable) y los **datos** (tu configuración, la memoria del agente, que deben persistir). Gracias al volumen, puedes borrar y recrear el contenedor mil veces — para actualizarlo, para arreglar un error — sin perder nunca tu configuración. Es la diferencia entre cambiar el reproductor de DVD (el contenedor) y conservar el DVD (tus datos).

Consejo de copia de seguridad: como los volúmenes guardan tus datos en una carpeta de tu Windows (por ejemplo `C:\Users\fernando\.openclaw`), hacer una copia de seguridad de todo tu asistente es tan simple como copiar esa carpeta a otro sitio.

7.5. El puerto: cómo hablar con el contenedor

Un contenedor está aislado, también en lo que respecta a la red. Para poder acceder a la aplicación que corre dentro (por ejemplo, abrir el panel de OpenClaw en el navegador), hay que **publicar un puerto**: abrir una "ventana" entre el contenedor y tu ordenador. Se hace con `-p` :

```
-p 18789:18789
```

Esto conecta el puerto 18789 de tu ordenador con el puerto 18789 de dentro del contenedor. Así, cuando abres `http://localhost:18789` en el navegador, llegas a la aplicación de dentro.

8. Comandos de Docker explicados

Ahora que entiendes los conceptos, aquí tienes los comandos de Docker que usarás, cada uno explicado con su propósito y sus opciones.

8.1. Comprobar que Docker funciona

```
docker --version
docker compose version
```

Si ambos responden con un número de versión, Docker está instalado y listo. Si no, hay que instalar Docker Desktop y asegurarse de que está arrancado.

Docker Compose v2: es una herramienta que viene con Docker para arrancar contenedores desde un archivo de configuración (`docker-compose.yml`) en lugar de comandos largos. El "v2" indica que se usa con la sintaxis moderna `docker compose` (con espacio), no la antigua `docker-compose` (con guion). Para este proyecto no es imprescindible, pero es muy usado en desarrollo web, así que conviene conocerlo.

8.2. El comando estrella: docker run

Crea y arranca un contenedor a partir de una imagen. Es el comando más importante. Desglosamos uno completo:

```
docker run -d --name openclaw --restart unless-stopped \
-e NVIDIA_API_KEY=$env:NVIDIA_API_KEY \
-v ${HOME}\.openclaw:/home/node/.openclaw \
-p 18789:18789 \
imagen:latest gateway
```

Parte	Qué significa
<code>docker run</code>	Crea y arranca un contenedor.
<code>-d</code>	"Detached": lo deja corriendo en segundo plano, devolviéndote la terminal.
<code>--name openclaw</code>	Le pone un nombre fijo al contenedor, para referirte a él después.
<code>--restart unless-stopped</code>	Hace que Docker lo relance solo tras reiniciar el equipo (salvo que lo pares a mano).
<code>-e CLAVE=valor</code>	Define una variable de entorno dentro del contenedor (aquí, la clave de NVIDIA).
<code>-v origen:destino</code>	Monta un volumen: conecta una carpeta del host con una del contenedor.
<code>-p 18789:18789</code>	Publica el puerto para poder acceder desde el navegador.
<code>imagen:latest</code>	La imagen a partir de la cual se crea el contenedor.
<code>gateway</code>	El comando que se ejecuta dentro del contenedor al arrancar.

8.3. Gestionar contenedores

Comando	Qué hace
<code>docker ps</code>	Lista los contenedores que están corriendo ahora.
<code>docker ps -a</code>	Lista todos los contenedores, incluso los parados.
<code>docker start openclaw</code>	Arranca un contenedor parado.
<code>docker stop openclaw</code>	Detiene un contenedor en marcha.
<code>docker restart openclaw</code>	Lo reinicia (parar + arrancar).
<code>docker rm openclaw</code>	Borra un contenedor (debe estar parado). Con <code>-f</code> lo fuerza aunque esté corriendo.
<code>docker logs openclaw</code>	Muestra los mensajes (logs) que ha producido el contenedor.
<code>docker logs -f openclaw</code>	Igual, pero "sigue" los logs en vivo (Ctrl+C para salir).

8.4. Ejecutar comandos dentro de un contenedor: `docker exec`

Este comando es fundamental para tu caso, porque te permite manejar OpenClaw desde fuera. Ejecuta un comando **dentro** de un contenedor que ya está corriendo:

```
docker exec openclaw openclaw config get tools.web.search.provider
```

Se lee así: `docker exec` (ejecuta dentro) + `openclaw` (el nombre del contenedor) + `openclaw config get ...` (el comando a ejecutar dentro). El primer `openclaw` es el contenedor; el segundo es el programa.

¿Por qué `docker exec` es tan útil? Porque OpenClaw vive dentro del contenedor, pero tú escribes desde PowerShell (fuera). `docker exec` es el "puente": te deja dar órdenes a OpenClaw sin tener que entrar manualmente al contenedor. Cualquier comando de la documentación que empiece por `openclaw ...` lo ejecutas anteponiendo `docker exec openclaw`.

Para comandos interactivos (que te van preguntando cosas), añade `-it`:

```
docker exec -it openclaw bash
```

Eso te mete en una terminal Bash **dentro** del contenedor. El prompt cambiará. Para salir, escribe `exit`.

8.5. Limpiar imágenes y espacio

Comando	Qué hace
<code>docker images</code>	Lista las imágenes descargadas.
<code>docker rmi imagen</code>	Borra una imagen.
<code>docker system prune</code>	Limpia contenedores parados, redes sin usar y caché. Libera espacio.

PARTE III

OpenClaw

El protagonista. Qué es, cómo está construido por dentro, y cómo instalarlo paso a paso en Docker.

9. Qué es OpenClaw y para qué sirve

OpenClaw es una plataforma de código abierto que convierte un modelo de IA en un asistente personal autoalojado, accesible desde un chat, con memoria y herramientas.

9.1. La idea central

OpenClaw (licencia MIT, software libre) actúa como una **puerta de enlace** (gateway) entre un canal de chat —un panel web, WhatsApp, Telegram...— y un modelo de inteligencia artificial. En lugar de abrir el navegador para hablar con una IA en la web de una empresa, montas tu propio asistente que corre en tu equipo y al que hablas desde donde quieras.

9.2. Qué lo diferencia

Característica	Qué significa
Autoalojado	Corre en tu hardware. Nada sale de tu máquina salvo lo que configures.
Privado	El estado del agente se guarda localmente. Tus conversaciones son tuyas.
Multi-canal	Un solo gateway puede atender varios canales a la vez (web, Telegram, etc.).
Independiente del modelo	Funciona con NVIDIA, OpenAI, Anthropic, Google, modelos locales... Cambias de modelo sin perder nada.
Con memoria y personalidad	El agente recuerda cosas entre sesiones y tiene un carácter configurable.

¿Por qué montar tu propio asistente en vez de usar uno de una web? Por control y privacidad. Tú decides qué modelo usa, dónde se guardan los datos, qué herramientas tiene y cómo se comporta. Tus conversaciones no alimentan a nadie. Y como proyecto de aprendizaje, montar esto te enseña Docker, APIs, configuración de sistemas y resolución de problemas reales: justo las competencias que se valoran en desarrollo.

9.3. Un concepto clave: OpenClaw no es la IA

Esto es importante y a menudo se malinterpreta. OpenClaw **no piensa**: enruta. El que genera las respuestas es el **modelo** (que puede estar en NVIDIA, en tu propio equipo, etc.). OpenClaw es la infraestructura que conecta tu chat con ese modelo y le añade memoria, herramientas y personalidad. Distinguir "gateway" (OpenClaw) de "modelo" (la IA) es la base para entender todo lo demás.

10. Arquitectura: gateway, agente, canales y modelos

Para configurar y arreglar OpenClaw hay que tener un mapa mental de sus piezas. Son pocas y, una vez las ves claras, todo encaja — independientemente de dónde y cómo lo instales.

10.1. Las piezas

Pieza	Rol
Gateway	El motor central. Gestiona canales, sesiones, herramientas y el enrutamiento al modelo. Es lo que "arrancas", sea como proceso nativo o como contenedor.
Agente	La IA con personalidad, workspace y memoria. Puede haber varios.
Modelo	El LLM que genera las respuestas. El "cerebro".
Proveedor	El servicio que sirve el modelo (NVIDIA, OpenAI, Anthropic, Google, Ollama local...). Define URL y clave.
Canal	La vía de entrada: panel web, WhatsApp, Telegram...
Workspace	La carpeta con los archivos que definen la personalidad y memoria del agente.
Sesión	Una conversación con continuidad de historial. Se reinicia con <code>/new</code> .

10.2. El recorrido de un mensaje

1. **Entrada:** el canal (el panel web, por ejemplo) recibe tu mensaje y lo envía al gateway.
2. **Enrutamiento:** el gateway comprueba quién eres (autenticación) y a qué agente dirigir el mensaje.
3. **Contexto:** el agente carga su workspace (personalidad, memoria) y el historial de la sesión.
4. **Modelo:** el mensaje, junto con todo ese contexto, se envía al modelo de IA.
5. **Herramientas:** si el modelo necesita buscar en internet o leer un archivo, llama a una herramienta.
6. **Respuesta:** el modelo genera la respuesta y el gateway te la devuelve por el canal.
7. **Memoria:** si hay automatizaciones activas, se guarda lo relevante en la memoria.

Nota: esta arquitectura no cambia según cómo instales OpenClaw. Lo único que cambia entre métodos es **dónde vive el proceso del gateway** y **dónde quedan guardados los archivos** — el resto del comportamiento es idéntico.

PARTE IV

Elige tu método de instalación

OpenClaw se puede instalar de varias formas igualmente válidas. Este bloque cubre cada una completa: Node.js nativo, Docker (dos variantes) y servidor remoto.

11. Mapa de decisión: qué método y dónde

Antes de teclear nada, conviene decidir **dónde** va a vivir tu instalación y **cómo** vas a instalarla. Son dos decisiones independientes que se combinan.

11.1. Dónde puede vivir OpenClaw

Ubicación	Cuándo tiene sentido
Tu propio ordenador (Windows, Linux o macOS)	Uso personal; el equipo debe estar encendido para que el agente responda.
Una máquina virtual local (VM)	Quieres aislar el entorno de tu sistema principal, o practicas Linux en paralelo.
Un servidor remoto / VPS	Quieres que el agente esté disponible 24/7 sin depender de que tu ordenador esté encendido.

11.2. Cómo puede instalarse (independiente de dónde)

Método	Cuándo usarlo	Ventaja	Inconveniente
A — Node.js nativo	Quieres el flujo más directo, sin capas extra.	Sencillo, arranque inmediato.	Depende de tu versión de Node; se instala "en" tu sistema.
B — Docker, imagen preconstruida	Quieres lo mínimo: bajar y arrancar.	Rapidísimo, sin compilar.	Dependes de quien mantiene esa imagen.
C — Docker, imagen oficial + Compose	Quieres construir tú mismo la imagen desde el código fuente.	Control total, reproducible.	Más pasos iniciales (clonar, construir).
D — Servidor remoto	Cualquiera de los métodos anteriores, pero ejecutado en una máquina remota a la que accedes por SSH.	Disponibilidad 24/7, no depende de tu equipo.	Coste del servidor, gestión de seguridad remota.

Lo importante de entender ya: los métodos A, B y C son *formas de instalar el software*; el método D es *un lugar donde ejecutar cualquiera de los anteriores*. Por eso, en la práctica, las combinaciones reales son: "Node nativo en mi PC", "Docker en mi PC", "Node nativo en un servidor", "Docker en un servidor"... El "dónde" y el "cómo" se eligen por separado.

11.3. La regla que no cambia, sea cual sea tu elección

Toda la configuración y los datos de OpenClaw viven en una carpeta llamada `.openclaw`. **Esa carpeta existe físicamente en la máquina donde corre el proceso del gateway** — tu PC, tu VM, o el servidor remoto. Si instalas con Docker, la carpeta puede vivir en el sistema anfitrión (gracias a un volumen) o dentro del propio contenedor, según cómo lo configures; los detalles exactos de cada combinación se explican en el capítulo 20.

11.4. Requisitos comunes a todos los métodos

Requisito	Detalle
RAM	Mínimo 2 GB libres para el proceso del gateway.
Clave API	De un proveedor de modelos (NVIDIA, OpenAI, Anthropic, Google...) o una instalación local de Ollama.
Disco	~200 MB para OpenClaw, más espacio si usas modelos locales.
Conexión a internet	Salvo que uses un modelo 100% local, necesitas salida a internet para hablar con el proveedor del modelo.

12. Método A — Node.js nativo (cualquier sistema operativo)

La vía más directa: instalar OpenClaw como un paquete del sistema, sin ninguna capa de aislamiento. Funciona igual en Windows, Linux y macOS, con la única diferencia del comando de instalación inicial.

12.1. Qué es Node.js

Node.js es el entorno que ejecuta el propio OpenClaw (está construido en JavaScript). Necesitas tenerlo instalado antes de empezar: la versión recomendada es Node 24, con 22.19 o superior como mínimo soportado.

12.2. Instalar OpenClaw

macOS / Linux (incluida una VM Linux local o un servidor remoto por SSH):

```
curl -fsSL https://openclaw.ai/install.sh | bash
```

Windows (PowerShell):

```
iwr -useb https://openclaw.ai/install.ps1 | iex
```

Cualquier plataforma (vía npm, si ya tienes Node instalado):

```
npm install -g openclaw@latest
```

¿Por qué `npm install -g` ? La `-g` significa "global": instala OpenClaw disponible como comando `openclaw` en todo el sistema, no solo en una carpeta concreta. Por eso después puedes escribir `openclaw` desde cualquier ruta.

12.3. Configuración inicial (onboarding)

```
openclaw onboard
```

Crea el archivo `~/.openclaw/openclaw.json` e inicializa el workspace. Las preguntas se detallan en el capítulo 16, comunes a todos los métodos.

Opcionalmente, instala el gateway como servicio del sistema para que arranque solo:

```
openclaw onboard --install-daemon
```

12.4. Arrancar y verificar

```
openclaw gateway start      # arranca en segundo plano (servicio)
openclaw gateway status    # comprueba que escucha en el puerto 18789
openclaw dashboard         # abre el panel web en el navegador
```

Diferencia clave: `openclaw gateway start` arranca en segundo plano (como un servicio); `openclaw gateway` (sin `start`) lo arranca en primer plano, mostrando los logs en la terminal hasta que lo cortes. Esto es importante de recordar al comparar con el método Docker, donde el comportamiento es justo al revés.

12.5. Si instalas Node nativo en un servidor remoto

El procedimiento es idéntico, pero ejecutado dentro de una sesión SSH:

```
ssh usuario@tu-servidor
curl -fsSL https://openclaw.ai/install.sh | bash
openclaw onboard --install-daemon
```

Cuidado: si cierras la sesión SSH y el gateway no quedó instalado como servicio (`--install-daemon`) ni arrancado en segundo plano, el proceso puede detenerse al cerrar la terminal. Usa siempre `gateway start` o el modo daemon para instalaciones en servidor.

13. Método B — Docker con imagen preconstruida

Aquí OpenClaw corre dentro de un **contenedor**: una caja aislada que ya incluye Node y todas las dependencias. Es la vía más rápida dentro de Docker: descargas una imagen ya construida por otra persona y la ejecutas directamente, sin compilar nada.

¿Por qué Docker en general? Con Docker, **no instalas OpenClaw en tu sistema**: vive en una caja sellada. Si algo se rompe, borras el contenedor y empiezas de nuevo sin haber tocado el resto de tu equipo. Es la opción más limpia para experimentar.

13.1. Requisitos de este método

Necesitas Docker Desktop (Windows/macOS) o Docker Engine (Linux) instalado y en marcha. En Windows, Docker usa WSL2 por debajo (capítulo 6); en Linux, Docker corre de forma nativa.

13.2. Paso 1 — Conseguir la clave de tu proveedor de modelos

Antes de nada, consigue una clave API del proveedor que vayas a usar (ver Parte V). Guárdala a mano.

13.3. Paso 2 — Bajar la imagen

```
docker pull ghcr.io/<publicador>/openclaw-docker:latest
```

13.4. Paso 3 — Definir la clave como variable de entorno

```
# PowerShell (Windows)
$env:MI_PROVEEDOR_API_KEY = "tu-clave-real"

# Bash (Linux/macOS)
export MI_PROVEEDOR_API_KEY="tu-clave-real"
```

13.5. Paso 4 — Onboarding

En PowerShell (Windows):


```
docker run -it --rm \
-e MI_PROVEEDOR_API_KEY=$env:MI_PROVEEDOR_API_KEY \
-v ${HOME}\.openclaw:/home/node/.openclaw \
-v ${HOME}\.openclaw\workspace:/home/node/.openclaw/workspace \
ghcr.io/<publicador>/openclaw-docker:latest onboard
```

En Bash (Linux/macOS, o un servidor remoto):

```
docker run -it --rm \
-e MI_PROVEEDOR_API_KEY=$MI_PROVEEDOR_API_KEY \
-v $HOME/.openclaw:/home/node/.openclaw \
-v $HOME/.openclaw/workspace:/home/node/.openclaw/workspace \
ghcr.io/<publicador>/openclaw-docker:latest onboard
```

El volumen (`-v`) es la pieza clave: "saca" los datos del contenedor y los guarda en la máquina anfitriona (sea tu PC, tu VM, o el servidor remoto). Sin esto, borrar el contenedor borraría también tu configuración.

13.6. Paso 5 — Arrancar el gateway

PowerShell:

```
docker run -d \
--name openclaw --restart unless-stopped \
-e MI_PROVEEDOR_API_KEY=$env:MI_PROVEEDOR_API_KEY \
-v ${HOME}\.openclaw:/home/node/.openclaw \
-v ${HOME}\.openclaw\workspace:/home/node/.openclaw/workspace \
-p 18789:18789 \
ghcr.io/<publicador>/openclaw-docker:latest gateway
```

Bash:

```
docker run -d \
--name openclaw --restart unless-stopped \
-e MI_PROVEEDOR_API_KEY=$MI_PROVEEDOR_API_KEY \
-v $HOME/.openclaw:/home/node/.openclaw \
-v $HOME/.openclaw/workspace:/home/node/.openclaw/workspace \
-p 18789:18789 \
ghcr.io/<publicador>/openclaw-docker:latest gateway
```

Detalle crítico: se usa `gateway` (primer plano), no `gateway start` (segundo plano). Si el proceso principal del contenedor pasa a segundo plano, Docker considera terminado el contenedor y lo apaga.

13.7. Paso 6 — El bind del gateway

Por defecto, tras el onboarding, el gateway puede quedar escuchando solo en "loopback" (127.0.0.1), que dentro del contenedor es **el propio contenedor**, no la máquina desde la que accedes. El resultado: el navegador no llega y ves `ERR_EMPTY_RESPONSE`.

```
docker exec openclaw openclaw config set gateway.bind lan
docker restart openclaw
```

13.8. Paso 7 — Acceder al panel

```
docker exec openclaw openclaw dashboard --no-open
```

Esto imprime la URL con un token de acceso. Si el contenedor corre en tu propio equipo, abres esa URL directamente en tu navegador. Si corre en un servidor remoto, normalmente necesitas un túnel SSH para llegar al puerto de forma segura (ver capítulo 15).

14. Método C — Docker con imagen oficial y Compose

La vía oficial completa: clonas el código fuente de OpenClaw y construyes tú mismo la imagen, en vez de depender de una ya publicada por terceros.

14.1. Cuándo elegir este método en vez del B

Elige el Método B (imagen preconstruida) si...	Elige el Método C (construir tú mismo) si...
Quieres arrancar en minutos, sin compilar.	Quieres control total sobre la versión exacta del código.
Te basta con confiar en quien mantiene la imagen.	Prefieres auditar y construir desde la fuente oficial.

14.2. Paso 1 — Clonar el repositorio

```
git clone https://github.com/openclaw/openclaw.git
cd openclaw
```

14.3. Paso 2 — Construir y configurar (script todo-en-uno)

```
./docker-setup.sh
```

Este script construye la imagen, ejecuta el onboarding, arranca el gateway vía Docker Compose y genera el token de acceso, guardándolo en un archivo `.env`.

14.4. Flujo manual equivalente, paso a paso

```
docker build -t openclaw:local -f Dockerfile .
docker compose run --rm openclaw-cli onboard
docker compose up -d openclaw-gateway
```

Dos contenedores con roles distintos: `docker compose run --rm openclaw-cli onboard` lanza un contenedor **temporal** (`--rm` lo borra al terminar) solo para configurar. `docker compose up -d openclaw-gateway` arranca el gateway de forma **persistente**.

14.5. Acceder al panel

```
# Abre http://127.0.0.1:18789/ en el navegador
# Si necesitas de nuevo la URL con token:
docker compose run --rm openclaw-cli dashboard --no-open
```

La configuración y el workspace se escriben en el host (sea tu equipo o el servidor donde clonaste el repositorio), en `~/.openclaw/` y `~/.openclaw/workspace` , por lo que persisten aunque borres y recrees los contenedores.

15. Método D — Servidor remoto / VPS

Cualquiera de los tres métodos anteriores puede ejecutarse en un servidor remoto en lugar de tu equipo local. La diferencia no está en cómo se instala OpenClaw, sino en cómo accedes a él desde fuera.

15.1. Por qué un servidor remoto

Un servidor remoto (VPS) está encendido permanentemente, así que tu asistente responde 24 horas al día sin depender de que tu propio ordenador esté arrancado. A cambio, asumes el coste del servidor y la responsabilidad de mantenerlo seguro frente a accesos no autorizados desde internet.

15.2. Conectarte al servidor

```
ssh usuario@la-ip-de-tu-servidor
```

Una vez dentro, instalas OpenClaw exactamente como en el Método A o el Método B/C, según prefieras — los comandos son idénticos a los de Linux, porque la mayoría de servidores VPS corren Linux.

15.3. Acceder de forma segura al panel desde tu equipo

El panel web normalmente conviene mantenerlo escuchando solo en `localhost` del servidor (es decir, sin exponerlo directamente a internet), y acceder a través de un **túnel SSH** que "trae" ese puerto remoto hasta tu máquina local de forma cifrada:

```
ssh -L 18789:localhost:18789 usuario@la-ip-de-tu-servidor
```

Con ese túnel abierto, accedes desde tu propio navegador exactamente igual que si OpenClaw estuviera en tu equipo:

```
http://localhost:18789
```

¿Por qué un túnel y no abrir el puerto directamente? Abrir el puerto 18789 a todo internet expone el panel a cualquiera que lo encuentre escaneando direcciones IP. Un túnel SSH cifra la conexión y la limita a quien tenga tus credenciales SSH — mucho más seguro para administración personal.

15.4. La alternativa: exposición controlada

Si necesitas que otras personas accedan sin pasar por SSH, las opciones más seguras son una VPN de malla como Tailscale (capítulo 27) o un proxy inverso con HTTPS y autenticación propia — nunca exponer el puerto en crudo sin protección adicional.

15.5. Mantener el gateway vivo tras cerrar la sesión SSH

Si instalaste con el Método A (Node nativo), usa el modo daemon (`--install-daemon` o `gateway start`), que deja el proceso corriendo como servicio del sistema. Si usaste Docker, el contenedor con `--restart unless-stopped` sigue vivo independientemente de tu sesión SSH — es una de las ventajas de los contenedores en este escenario.

16. El onboarding pregunta a pregunta (todos los métodos)

El asistente de configuración inicial hace las mismas preguntas sea cual sea el método de instalación elegido (A, B o C) y la ubicación (local o remota). Aquí están todas explicadas, con la respuesta recomendada y el porqué.

Pregunta	Qué elegir	Por qué
Aviso de "personal por defecto"	Yes	Confirmas que es de uso personal. Si fuera compartido haría falta blindaje extra.
Setup mode	Manual setup	Da control sobre el proveedor de modelo, en vez de valores por defecto.
What to set up	Local gateway	El gateway corre en la máquina donde ejecutas el comando (tu equipo o el servidor remoto).
Workspace directory	El valor por defecto	Es la ruta interna estándar; en Docker, ya está mapeada al volumen.
Model / auth provider	El proveedor elegido (Parte V)	Detecta tu clave si la exportaste antes.
Default model	Keep current (o elegir otro)	Puedes empezar con el por defecto y cambiarlo luego.
Gateway port	18789	El puerto estándar.
Gateway bind	Loopback (se ajustará a <code>lan</code> después si usas Docker)	En instalación nativa, loopback es correcto desde el principio; en Docker se corrige tras el onboarding (capítulo 13).
Access protection	Token	Protege el panel con un token aleatorio seguro.
Gateway token	En blanco (para generar)	Deja que genere uno seguro automáticamente.
Set up a chat channel	No (de momento)	El panel web ya trae chat. Los canales como Telegram se añaden después.
Web search	Skip / DuckDuckGo	Se puede activar luego sin problema. DuckDuckGo no necesita clave.
Skills / plugins / hooks	Skip / lo mínimo	Se añaden cuando se necesiten.
Hatch your agent	Hatch later	Mejor arrancar el gateway aparte y hablar desde el panel.

Consejo: casi todas las respuestas "por defecto" o "Skip" son seguras para empezar. Lo único que conviene elegir con intención es el **proveedor de modelo**. Todo lo demás se ajusta después con calma, y de la misma forma sea cual sea tu método de instalación.

Sobre el "hatch": al final, el agente hace un "ritual de nacimiento" (bootstrap) donde le das nombre, le cuentas quién eres y tus preferencias. Esas respuestas se guardan en sus archivos de personalidad. Puedes hacerlo en el primer chat del panel, sea cual sea el método o la ubicación elegidos.

PARTE V

Modelos de IA

El cerebro del asistente. Qué es un modelo de lenguaje, cómo elegir un proveedor, y cómo conectarlo, sea cual sea.

17. Qué es un modelo de lenguaje y cómo se elige

El modelo es lo que de verdad "piensa" y genera las respuestas. OpenClaw solo lo conecta. Entender qué es y cómo se diferencian unos de otros te permite elegir bien.

17.1. Qué es un LLM

Un **modelo de lenguaje grande** (LLM, por *Large Language Model*) es un programa de IA entrenado con enormes cantidades de texto para predecir y generar lenguaje. No "busca" la respuesta en una base de datos: la genera palabra a palabra, basándose en lo que aprendió.

17.2. Parámetros: el "tamaño" del modelo

Los modelos se miden en **parámetros** (por ejemplo, "120B" = 120.000 millones). A grandes rasgos, más parámetros suele significar más capacidad de razonamiento, pero también más lentitud y más coste de cómputo.

¿Por qué no usar siempre el modelo más grande? Porque "más grande" implica "más lento", y en un servicio con límites generosos pero no ilimitados eso puede provocar que el modelo tarde tanto que el sistema corte la conexión por tiempo de espera. Para un asistente de chat, un modelo intermedio y ágil suele dar mejor experiencia que el gigante más potente pero lento. La decisión correcta equilibra **potencia, velocidad y fiabilidad**, no solo el tamaño.

17.3. Capacidades que conviene verificar

No todos los modelos sirven igual para todo. Una capacidad concreta y crítica es el **"tool calling"** (uso de herramientas): la habilidad del modelo para llamar correctamente a herramientas como la búsqueda web. Algunos modelos son excelentes conversando pero fallan al usar herramientas. Conviene **probar** que el modelo elegido hace bien lo que vas a necesitar.

Necesidad	Qué priorizar
Chat rápido y fluido	Un modelo ágil, buen manejo del idioma.
Razonamiento complejo (mates, lógica)	Un modelo potente, aunque sea más lento.
Búsqueda web / herramientas	Un modelo con buen "tool calling" comprobado.
Privacidad total / sin coste recurrente	Un modelo local vía Ollama o llama.cpp.

18. Conectar un proveedor de modelos

OpenClaw es independiente del modelo: puedes conectarlo a más de treinta proveedores distintos, desde los grandes nombres comerciales hasta modelos que corren enteramente en tu equipo. El procedimiento general es el mismo para todos; solo cambian el nombre de la variable de entorno y la URL base.

18.1. Proveedores habituales

Proveedor	Tipo	Requiere
Anthropic	Comercial (API)	Clave API o suscripción
OpenAI	Comercial (API)	Clave API
Google (Gemini)	Comercial (API)	Clave API
NVIDIA (NIM)	Comercial, con tier gratuito generoso	Clave API (gratuita) de build.nvidia.com
OpenRouter	Agregador de muchos modelos	Clave API
Ollama / llama.cpp / LM Studio	Local, sin coste recurrente	El modelo descargado en tu propio equipo

18.2. El patrón general de conexión

Para cualquier proveedor basado en API, el patrón es el mismo: obtienes una clave en el sitio del proveedor, la exportas como variable de entorno, y OpenClaw la detecta sola.

```
# PowerShell
$env:NOMBRE_PROVEEDOR_API_KEY = "tu-clave"

# Bash
export NOMBRE_PROVEEDOR_API_KEY="tu-clave"

# En Docker, se pasa al contenedor con -e:
-e NOMBRE_PROVEEDOR_API_KEY=$NOMBRE_PROVEEDOR_API_KEY
```

El nombre exacto de la variable depende del proveedor (por ejemplo, `ANTHROPIC_API_KEY`, `OPENAI_API_KEY`, `NVIDIA_API_KEY`) — consulta la documentación del proveedor concreto que elijas para confirmar el nombre exacto.

18.3. Listar y cambiar de modelo

Desde el chat del panel, puedes ver los modelos disponibles para un proveedor ya conectado y cambiarlos en caliente:

```
/models <proveedor>           # lista los modelos del proveedor
/model <proveedor>/<modelo>    # cambia el modelo de la sesión
```

Cuidado con los IDs de modelo: el identificador debe escribirse **exactamente** como aparece en la lista de `/models`. Un ID mal escrito da el error `model not allowed`. Y los nombres de modelo cambian con frecuencia, así que un ID de memoria puede quedar obsoleto — cópialo siempre de la lista actual.

18.4. Permitir un modelo en la configuración

Para que un modelo se pueda usar, debe estar en la lista de modelos permitidos del archivo de configuración (`agents.defaults.models`). Ejemplo genérico (sustituye por tu proveedor real):

```
"models": {
  "<proveedor>/<modelo-rapido>": {},
  "<proveedor>/<modelo-potente>": {}
},
"model": { "primary": "<proveedor>/<modelo-rapido>" }
```

18.5. Modelos locales: una nota aparte

Si eliges un proveedor local (Ollama, llama.cpp, LM Studio), no necesitas clave API ni conexión a internet para la inferencia en sí — solo necesitas tener ese servicio corriendo en la misma máquina (o una accesible en red) donde está el gateway de OpenClaw, y apuntar la configuración a su URL local en vez de a una URL de internet.

19. Tokens, contexto y velocidad

En el panel verás un indicador del tipo "9% context used 18.6k / 202.8k". Entender qué significa te ayuda a usar el asistente de forma más eficiente, sea cual sea el proveedor que uses.

19.1. Qué es un token

Un **token** es la unidad mínima con la que el modelo lee y escribe texto. No es exactamente una palabra: suele ser un trozo de palabra. En español, un token equivale aproximadamente a 0,75 palabras (o unas 4 letras).

19.2. La ventana de contexto

La **ventana de contexto** es la "memoria de trabajo" del modelo en una conversación: todo lo que puede tener presente a la vez. Incluye tu mensaje, los anteriores, las respuestas y los archivos del workspace que se cargan al inicio. Cada modelo tiene su propio tamaño de ventana — consulta la documentación del proveedor o usa `/models` para verlo.

19.3. Qué pasa cuando se llena

Cuando se acerca al máximo, OpenClaw ejecuta una pasada silenciosa de "**compaction**": guarda lo importante en memoria y resume el resto, sin perder la información relevante.

Consejo: cuando cambies de tema, escribe `/new` para abrir una sesión limpia. Esto baja el contador de contexto y le da al modelo espacio fresco, manteniendo intactas la personalidad y la memoria del agente.

Sobre el coste: con un proveedor de tier gratuito, los tokens no cuestan dinero; el indicador importa solo por la memoria del modelo. Con proveedores de pago, en cambio, cada token tiene un coste, y ahí el contador sí afecta a la factura.

PARTE VI

Configuración y sistema de archivos

Dónde vive cada cosa según tu método de instalación, cómo se configura técnicamente el sistema, y cómo se define la personalidad del agente.

20. Dónde vive .openclaw según el método elegido

Esta es la pregunta que conecta todo lo anterior: sea cual sea el método de instalación y la ubicación, toda la configuración y el estado de OpenClaw viven en una carpeta llamada `.openclaw`. Lo que cambia es **en qué máquina y en qué ruta exacta** aparece esa carpeta.

La regla de oro, para no perderse nunca: la carpeta `.openclaw` existe físicamente en la máquina donde corre el **proceso del gateway** — no en la máquina desde la que accedes al panel con el navegador (si son distintas). Si el gateway corre en un servidor remoto, `.openclaw` está en ese servidor, no en tu portátil. Si corre en un contenedor Docker sin volúmenes, está dentro del contenedor, no en tu sistema operativo anfitrión.

20.1. Tabla comparativa completa

Método y ubicación	Ruta de .openclaw	Notas
Node nativo en Windows	<code>C:\Users\<usuario>\.openclaw</code>	Accesible directamente con el Explorador de Windows.
Node nativo en Linux o macOS	<code>~/.openclaw</code> (p. ej. <code>/home/usuario/.openclaw</code> o <code>/Users/usuario/.openclaw</code>)	<code>~</code> es un atajo de la terminal para "mi carpeta personal".
Docker en Windows (con volumen)	<code>C:\Users\<usuario>\.openclaw</code> en el host; <code>/home/node/.openclaw</code> dentro del contenedor	Son la misma carpeta vista desde dos sitios — el volumen los conecta.
Docker en Linux/macOS (con volumen)	<code>~/.openclaw</code> en el host; <code>/home/node/.openclaw</code> dentro del contenedor	Mismo principio que en Windows.
Docker sin volumen (cualquier sistema)	Solo dentro del contenedor, en <code>/home/node/.openclaw</code>	Se pierde al borrar el contenedor. No recomendado salvo pruebas desechables.
Cualquier método en un servidor remoto	La ruta equivalente de Linux (<code>~/.openclaw</code> o, en Docker, el volumen mapeado), pero en el servidor , no en tu equipo local	Para verla o editarla, necesitas una conexión SSH al servidor; tu navegador local solo ve el panel web, no el sistema de archivos remoto.

20.2. Cómo saber, en tu caso concreto, dónde está

Si tienes dudas, estos comandos te lo confirman sin ambigüedad:

```
# Si instalaste con Node nativo (en la propia máquina donde corre el gateway):
openclaw config get agents.defaults.workspace

# Si instalaste con Docker, primero entra al contenedor:
docker exec -it openclaw openclaw config get agents.defaults.workspace

# Para ver si esa ruta del contenedor está, además, mapeada a tu sistema (es decir, si
hay volumen):
docker inspect openclaw --format '{{json .Mounts}}'
```

El último comando es el que de verdad responde a tu pregunta: si la lista que devuelve está vacía, no hay ningún volumen, y la carpeta solo existe dentro del contenedor. Si devuelve una ruta del host, ahí es exactamente donde puedes editar los archivos directamente.

20.3. Editar archivos según dónde estén

Situación	Cómo editar
Node nativo, en tu propio equipo	Cualquier editor de texto normal (Bloc de notas, VS Code, nano...), apuntando directamente a la ruta.
Docker con volumen, en tu propio equipo	Igual que arriba: edita la ruta del host. Los cambios se reflejan dentro del contenedor al instante (es la misma carpeta).
Docker sin volumen	<code>docker exec -it openclaw bash</code> , y editar con un editor de terminal (<code>nano</code> , <code>vi</code>) dentro del contenedor.
Cualquier método en un servidor remoto	Conéctate primero por SSH (<code>ssh usuario@servidor</code>) y edita allí, o usa la edición remota de tu editor (por ejemplo, la extensión "Remote - SSH" de VS Code).

Cuidado con confundir "donde veo el panel" con "donde están los archivos". Puedes abrir el panel web de OpenClaw desde el navegador de cualquier ordenador (incluido tu móvil), pero eso no significa que los archivos estén ahí. El navegador solo muestra una interfaz remota; los archivos siguen viviendo exclusivamente en la máquina donde corre el gateway.

20.4. La estructura interna (igual en todos los casos)

Una vez localizada la carpeta correcta para tu instalación, su contenido es siempre el mismo:


```

.openclaw/
├── openclaw.json      → Configuración técnica (modelo, gateway, herramientas)
├── workspace/        → El "cerebro" del agente (personalidad y memoria)
│   ├── SOUL.md       → Personalidad, tono y reglas de conducta e idioma
│   ├── IDENTITY.md   → Nombre, emoji y "vibe" del agente
│   ├── USER.md       → Información y preferencias del usuario
│   ├── MEMORY.md     → Memoria a largo plazo (hechos duraderos)
│   ├── AGENTS.md     → Instrucciones operativas (reglas de funcionamiento)
│   ├── TOOLS.md      → Notas sobre el entorno y las herramientas
│   └── memory/       → Diario por días (notas y resúmenes de cada sesión)
│       └── AAAA-MM-DD.md

```

20.5. Dos niveles bien diferenciados

- **openclaw.json** → la configuración *técnica*: qué modelo se usa, en qué puerto escucha el gateway, qué herramientas están activas.
- **workspace/** → la *identidad* del agente: quién es, cómo habla, qué recuerda. Son archivos Markdown **independientes del modelo y del método de instalación**.

¿Por qué esto importa especialmente al cambiar de método o de máquina? Porque significa que **migrar tu agente de un método a otro, o de una máquina a otra, es tan simple como copiar la carpeta `.openclaw` entera** al nuevo destino y apuntar un gateway nuevo a ella. La personalidad, la memoria y la configuración viajan juntas, sin depender de si el origen era Node nativo y el destino es Docker, o viceversa. El capítulo 30 detalla este proceso.

21. openclaw.json: la configuración técnica

Es el archivo central de configuración. Usa formato JSON5 (admite comentarios y comas finales). El gateway lo vigila y aplica la mayoría de cambios en caliente.

21.1. Las secciones principales

Sección	Qué controla
<code>agents</code>	Workspace, modelo primario, modelos permitidos.
<code>gateway</code>	Puerto, bind, autenticación.
<code>channels</code>	Los canales de mensajería y su política de acceso.
<code>tools</code>	Herramientas y búsqueda web.
<code>models.providers</code>	Proveedores personalizados (URL, clave, timeout).

21.2. Qué requiere reinicio y qué no

Cambias...	¿Reinicio?
Canales, agentes, modelos, herramientas, hooks	No (se aplica en caliente)
<code>gateway.*</code> (puerto, bind, auth)	Sí
plugins	Sí

21.3. Editar la configuración: dos formas

- **Con comandos** (lo normal): `openclaw config set <clave> <valor>` (en Docker, antepón `docker exec openclaw`).
- **A mano** (cuando los comandos fallan): editar el archivo directamente con un editor de texto, en la ubicación que corresponda según el capítulo 20.

Cuándo editar a mano: si la configuración queda en estado inválido, los comandos `config set` se niegan a tocarla. En ese caso, la única salida es abrir el `openclaw.json` con un editor de texto, corregir el valor problemático, guardar y reiniciar.

22. El workspace: la personalidad del agente

El workspace es donde vive el "alma" del agente. Son archivos de texto Markdown que el modelo lee al inicio de cada sesión, sea cual sea el método y la ubicación de tu instalación.

22.1. Los archivos y su función

Archivo	Función
<code>SOUL.md</code>	Personalidad, tono, voz, idioma, límites de conducta.
<code>IDENTITY.md</code>	Nombre, emoji y "vibe" del agente. Se genera en el bootstrap inicial.
<code>USER.md</code>	Lo que el agente sabe de ti: nombre, intereses, preferencias.
<code>MEMORY.md</code>	Memoria a largo plazo: hechos duraderos y decisiones, en forma curada.
<code>AGENTS.md</code>	Instrucciones operativas: reglas de cómo debe comportarse.
<code>TOOLS.md</code>	Notas sobre el entorno y las herramientas locales.
<code>memory/AAAA-MM-DD.md</code>	Diario por día con notas y resúmenes de cada sesión.

22.2. Cómo se le da memoria

OpenClaw "recuerda" escribiendo archivos de texto. Para que recuerde algo, basta con decírselo en el chat ("recuerda que prefiero explicaciones con ejemplos"): el agente lo escribe en el archivo apropiado, y lo tendrá presente en futuras sesiones.

22.3. El idioma y los modelos multilingües

Algunos modelos, sobre todo los multilingües, a veces "cuelan" una palabra de otro idioma en medio de una frase. Se reduce reforzando la regla de idioma en el `SOUL.md`, con instrucciones explícitas.

¿Por qué el SOUL.md a menudo se escribe en inglés? Las instrucciones de *comportamiento* (tono, reglas, idioma) tienden a seguirse de forma algo más consistente en inglés, ya que los modelos se entrenan con mucho más texto en ese idioma. Esto no impide que el agente responda en español: una cosa es el idioma de las instrucciones internas y otra el de las respuestas.

23. Búsqueda web y herramientas

Las herramientas amplían lo que el agente puede hacer: buscar en internet, leer archivos, ejecutar comandos.

23.1. Activar la búsqueda web con DuckDuckGo

DuckDuckGo viene integrado y no necesita clave API:

```
openclaw config set tools.web.search.provider duckduckgo
openclaw config set tools.web.search.enabled true
```

En Docker, antepón `docker exec openclaw` a cada comando, y reinicia el contenedor tras el cambio.

23.2. Probar que funciona

En el chat, pídele algo que requiera buscar: "busca en internet las novedades de tu lenguaje de programación favorito". Si responde con información actual, funciona.

Cuidado — no todos los modelos saben usar herramientas: la búsqueda puede estar bien configurada y aun así fallar si el modelo activo no maneja bien el "tool calling". Si el agente se queda "pensando" sin completar la búsqueda, prueba a cambiar a un modelo distinto.

PARTE VII

Operación y resolución de problemas

El día a día con cualquier método: cómo encender y apagar, cómo leer los logs, y un catálogo de problemas reales con sus soluciones.

24. Operar el gateway en el día a día (según el método)

Una vez montado, el uso cotidiano se reduce a unos pocos comandos — pero son distintos según cómo lo instalaste.

24.1. Encender, apagar, reiniciar

Acción	Node nativo	Docker
Encender	<code>openclaw gateway start</code>	<code>docker start openclaw</code>
Apagar	<code>openclaw gateway stop</code>	<code>docker stop openclaw</code>
Reiniciar	<code>openclaw gateway restart</code>	<code>docker restart openclaw</code>
Ver si está corriendo	<code>openclaw gateway status</code>	<code>docker ps</code>

Nota: con el daemon instalado (Node nativo) o con `--restart unless-stopped` (Docker), el gateway se levanta solo cuando reinicias el equipo o el servidor (siempre que el motor correspondiente esté en marcha).

24.2. Acceder al panel

Método	Comando para obtener la URL con token
Node nativo	<code>openclaw dashboard --no-open</code>
Docker	<code>docker exec openclaw openclaw dashboard --no-open</code>

Si el gateway corre en un servidor remoto, recuerda abrir primero el túnel SSH (capítulo 15) antes de visitar la URL en tu navegador local.

24.3. Ejecutar cualquier comando de OpenClaw, sea cual sea el método

Cualquier comando `openclaw ...` de este manual se traduce así según el método:

```
# Node nativo: directo
openclaw config get tools.web.search.provider

# Docker: con el prefijo docker exec
docker exec openclaw openclaw config get tools.web.search.provider
```

¿Por qué **docker exec** ? Ejecuta un comando **dentro** de un contenedor que ya está corriendo. El primer **openclaw** es el nombre del contenedor; el segundo es el programa. Es el "puente" entre tu terminal y el proceso que vive dentro de la caja aislada.

24.4. Comandos dentro del chat (iguales en todos los métodos)

Comando	Efecto
<code>/new</code>	Inicia una sesión nueva (recarga los archivos del workspace).
<code>/model</code> · <code>/models</code>	Cambia el modelo / lista los disponibles.
<code>/context list</code>	Muestra los archivos de contexto cargados y su tamaño.
<code>/help</code>	Ayuda de comandos.

25. Leer logs y diagnosticar

Los logs son los mensajes que el gateway va dejando mientras funciona. Aprender a leerlos convierte un problema de horas en uno de minutos, sea cual sea tu método de instalación.

25.1. Ver los logs

Método	Comando
Node nativo	<code>openclaw gateway logs</code> (o el archivo de log indicado en <code>openclaw gateway status</code>)
Docker	<code>docker logs openclaw</code> / <code>docker logs -f openclaw</code> (en vivo) / <code>docker logs --tail 40 openclaw</code> (últimos 40)

25.2. Qué buscar en los logs

- `error`, `failed` → algo salió mal; suele venir con la razón.
- `listening`, `ready` → el gateway arrancó bien.
- `AbortError`, `timeout` → el modelo tardó demasiado.
- `version mismatch` → desajuste de versiones, causa de muchos errores aparentemente inconexos.
- `model not allowed` → el modelo no está en la lista de permitidos.

25.3. El comando doctor

```
openclaw doctor          # diagnostica
openclaw doctor --fix    # intenta reparar
openclaw config validate # valida la configuración
```

En Docker, antepón `docker exec openclaw` a cada uno de estos comandos.

26. Catálogo de problemas reales y soluciones

Estos son los problemas más frecuentes al montar OpenClaw, con su causa y su solución. La mayoría no son errores del usuario: son desajustes de configuración o de entorno.

26.1. El panel no carga (ERR_EMPTY_RESPONSE) — específico de Docker

Causa: el gateway escucha solo en el loopback interno del contenedor, que no es la máquina desde la que accedes.

```
docker exec openclaw openclaw config set gateway.bind lan
docker restart openclaw
```

Aprendizaje: entender la diferencia entre el `localhost` del host y el del contenedor es fundamental en redes con Docker.

26.2. Configuración inválida que bloquea el arranque

Causa: un valor que el validador no reconoce deja toda la config inválida, y los propios comandos `config set` se niegan a modificarla: un círculo vicioso.

Solución: editar el archivo `openclaw.json` a mano (en la ubicación que corresponda según el capítulo 20) para eliminar la clave problemática, validar y reiniciar.

26.3. Timeouts al usar un modelo demasiado grande

Causa: el modelo más potente es demasiado lento para el límite de tiempo de espera configurado, y no emite respuesta antes de que salte el "watchdog" de inactividad.

Solución: cambiar a un modelo más ágil, o subir el timeout específico del proveedor (`models.providers.<proveedor>.requestTimeout`) en vez de un ajuste global.

26.4. Un modelo que no sabe usar herramientas

Causa: algunos modelos dan buenas respuestas de chat pero fallan al llamar a herramientas, generando un formato que el sistema no puede interpretar.

Solución: usar un modelo que maneje bien el "tool calling" para las tareas que requieren herramientas.

26.5. Desajuste de versiones

Causa: en instalaciones por Docker con imágenes de terceros, la CLI y el gateway pueden traer versiones distintas, provocando que ciertas claves de configuración (y plugins) sean rechazadas.

Solución: actualizar la imagen para alinear versiones, o rodear el problema usando solo las claves que la versión activa acepta (por ejemplo, sustituir un plugin incompatible por una alternativa integrada).

26.6. Otros mensajes frecuentes

Mensaje	Significado	Qué hacer
<code>model not allowed</code>	El modelo no está en la lista de permitidos.	Añadirlo a <code>agents.defaults.models</code> .
<code>thinkingLevel not supported</code>	El modelo solo admite el razonamiento en "off".	Poner el "thinking" en Off en el chat.
<code>unauthorized / pairing required</code>	Falta el token o aprobar el dispositivo.	Obtener la URL con token y aprobar el dispositivo.

27. Seguridad

Un asistente autónomo con acceso a herramientas implica riesgo por diseño. Estas son las medidas básicas, especialmente importantes si tu gateway corre en un servidor remoto accesible desde internet.

- **Autenticación por token:** el panel exige un token. Es la opción por defecto y la recomendada.
- **Bind loopback vs lan:** en local, loopback es lo más seguro; en Docker necesitas `lan`, pero el puerto solo debe publicarse hacia tu propia red, no hacia internet directamente.
- **En un servidor remoto, nunca expongas el puerto del panel sin protección:** usa un túnel SSH o una VPN de malla (como Tailscale) en vez de abrirlo directamente a internet.
- **No exponer secretos:** nunca subas el `openclaw.json` (tiene el token) ni datos personales a un repositorio público.
- **Allowlists de canales:** si conectas WhatsApp o Telegram, controla quién puede hablar con el agente.

Regla de oro: mantén el acceso restringido a lo que de verdad necesitas. Un asistente personal accesible solo desde tu propia red (o por túnel SSH) es seguro. En cuanto lo abres directamente a internet sin protección adicional, la superficie de ataque crece considerablemente.

PARTE VIII

Temas avanzados

Cuando ya domines lo básico: varios agentes, hablar desde el móvil, y mover tu instalación entre métodos o máquinas.

28. Multi-agente

OpenClaw puede alojar varios agentes a la vez, cada uno con su propia personalidad, memoria y hasta su propio modelo. Es una función potente, pero no siempre necesaria.

28.1. Qué es un segundo agente

Cada agente es un "cerebro" completamente aislado, con su propio workspace, su memoria y sus sesiones.

28.2. Cómo se crea

```
openclaw agents add <nombre>
```

En Docker: `docker exec -it openclaw openclaw agents add <nombre> .`

Esto crea una carpeta de workspace separada, con sus propios archivos de personalidad. En la configuración, los agentes se definen bajo `agents.list`, cada uno apuntando a su workspace, y se les puede asignar un modelo distinto.

28.3. ¿Lo necesitas?

¿Multi-agente o sesiones? Para separar *temas* dentro de tu uso diario NO necesitas otro agente: basta con `/new`. El multi-agente solo tiene sentido si quieres una *persona* genuinamente distinta y aislada. Para un usuario único, un solo agente con sesiones suele ser lo más sensato.

29. Canales: hablar desde el móvil

Hasta ahora hablas con el agente desde el navegador. Conectando un canal de mensajería, puedes hablarle desde el móvil, desde cualquier sitio.

29.1. La idea

Un **canal** es una vía de entrada. Además del panel web, OpenClaw soporta más de veinte canales: Telegram, WhatsApp, Discord, Slack, Signal, iMessage, SMS y muchos más.

Importante: aunque le hables desde el móvil, el proceso del gateway debe seguir corriendo en la máquina donde lo instalaste (encendida, con Docker activo si aplica). El canal es solo la "puerta" por la que llegan los mensajes; el gateway sigue viviendo donde tú lo pusiste.

29.2. Ejemplo: Telegram

A grandes rasgos: creas un bot en Telegram hablando con @BotFather, copias el token que te da, y lo añades a OpenClaw como canal. Luego emparejas tu cuenta con el bot.

30. Copias de seguridad, portabilidad y migración entre métodos

*Todo el "ser" de tu asistente —su configuración, su personalidad, su memoria— vive en una sola carpeta. Eso hace que respaldarlo, moverlo, e incluso **cambiar de método de instalación**, sea sencillo.*

30.1. Hacer una copia de seguridad

Localiza tu carpeta `.openclaw` según el capítulo 20, y cópiala a otro sitio (un disco externo, la nube, otro servidor).

Consejo: una buena práctica es guardar una copia del `workspace` de vez en cuando. Eso sí: no guardes el `openclaw.json` con el token en un sitio público.

30.2. Migrar entre máquinas, sin cambiar de método

Instala el mismo método (Node nativo o Docker) en la máquina destino, copia la carpeta `.openclaw` completa al mismo lugar relativo, y arranca el gateway nuevo apuntando a ella. Adopta toda la configuración y memoria sin pérdida.

30.3. Migrar de Node nativo a Docker (o viceversa)

Esto es posible precisamente porque la estructura interna de `.openclaw` es idéntica en ambos métodos (capítulo 20.4). El proceso:

1. Localiza tu carpeta `.openclaw` actual (Node nativo: `~/ .openclaw` ; en Windows, `C:\Users\<usuario>\.openclaw`).
2. Detén el gateway actual.
3. Si vas hacia Docker: monta esa misma carpeta como volumen al arrancar el contenedor nuevo (capítulo 13, Paso 5), usando la ruta exacta como origen del `-v` .
4. Si vas desde Docker hacia Node nativo: copia el contenido del volumen del host a `~/ .openclaw` , instala OpenClaw nativo, y arranca con `openclaw gateway` apuntando a esa misma carpeta.

Esto funciona porque OpenClaw no "sabe" ni le importa si está corriendo dentro de un contenedor o directamente sobre el sistema operativo — solo lee y escribe en la ruta de `.openclaw` que se le indique. El método de ejecución y los datos son capas independientes.

30.4. Migrar de tu equipo local a un servidor remoto

El mismo principio, añadiendo el paso de transferencia por red:

```
# Desde tu equipo local, copia la carpeta al servidor por SSH:  
scp -r ~/.openclaw usuario@tu-servidor:~/.openclaw
```

Luego instala OpenClaw en el servidor (Método A o B/C, capítulos 12-14) y arranca el gateway apuntando a esa carpeta ya poblada — el onboarding ni siquiera haría falta repetirlo, porque la configuración ya existe.

PARTE VIII

Git y GitHub

Cómo guardar la historia de tu trabajo y publicarlo para que otros —incluidos reclutadores— vean lo que has hecho.

28. Qué es el control de versiones

Git y GitHub son dos cosas distintas que suelen confundirse. Entender la diferencia es el primer paso para usarlos bien.

28.1. Git: la máquina del tiempo de tu código

Git es un sistema de *control de versiones*: un programa que registra los cambios que haces en tus archivos a lo largo del tiempo. Cada vez que guardas un "punto" (un *commit*), Git hace una foto del estado de tus archivos. Así puedes volver atrás, ver qué cambió y cuándo, y trabajar sin miedo a romper algo: siempre puedes regresar a una versión anterior.

28.2. GitHub: la nube para tus repositorios Git

GitHub es un servicio web donde puedes guardar tus repositorios de Git en la nube, compartirlos y mostrarlos. Git es la herramienta (vive en tu ordenador); GitHub es el sitio donde publicas lo que Git gestiona. Puedes usar Git sin GitHub, pero GitHub sin Git no tiene sentido.

	Git	GitHub
Qué es	Un programa en tu ordenador.	Un sitio web.
Función	Registrar versiones de tus archivos.	Alojar y compartir esos repositorios.
Necesita internet	No.	Sí.

28.3. Conceptos básicos de Git

Término	Qué es
Repositorio	La carpeta de tu proyecto, con todo su historial de versiones.
Commit	Una "foto" guardada del estado del proyecto, con una nota de qué cambió.
Push	Subir tus commits a GitHub.
Clone	Descargar un repositorio de GitHub a tu ordenador.
.gitignore	Un archivo que lista lo que Git debe ignorar (no subir).

29. Git y GitHub paso a paso

Dos formas de subir un proyecto a GitHub: la web (sin terminal) y la línea de comandos.

29.1. Crear el repositorio en GitHub

En github.com, botón "New", le pones nombre, eliges visibilidad (Public para que se vea), y lo creas. Para subir tu propio README y .gitignore, deja desmarcadas las opciones de "Add README" y "Add .gitignore" (los subes tú).

29.2. Subir por la web (la forma fácil)

En el repositorio, "Add file" → "Upload files", arrastras tus archivos, escribes un título de commit y "Commit changes". Sin tocar la terminal.

29.3. Subir por la terminal (practica Linux)

```
git init
git add .
git commit -m "Documentación del proyecto"
git branch -M main
git remote add origin https://github.com/usuario/repositorio.git
git push -u origin main
```

Comando	Qué hace
<code>git init</code>	Crea un repositorio Git en la carpeta actual.
<code>git add .</code>	Marca todos los archivos para el próximo commit.
<code>git commit -m "..."</code>	Guarda una foto con un mensaje descriptivo.
<code>git remote add origin ...</code>	Conecta tu repo local con el de GitHub.
<code>git push</code>	Sube los commits a GitHub.

Cuidado con la autenticación: al hacer `git push`, GitHub ya no acepta contraseña normal. Necesitas un *token de acceso personal* (Personal Access Token), que se genera en la configuración de GitHub. Es un paso de un solo uso, pero suele pillar de sorpresa la primera vez.

29.4. El .gitignore: tu red de seguridad

Este archivo le dice a Git qué NO subir. Es la herramienta que evita que publiques secretos por error. Un ejemplo para este proyecto:

```
.env  
openclaw.json  
**/credentials/  
workspace/USER.md  
workspace/MEMORY.md  
**/*.sqlite  
logs/
```

Importante — el `.gitignore` es visible, lo que protege no: el propio archivo `.gitignore` sí se ve en GitHub (es solo una lista de reglas). Pero los archivos que menciona **nunca se suben**, así que su contenido permanece privado. La lista es pública; los archivos listados, no.

30. Documentar el proyecto para un portfolio

Un proyecto bien documentado vale más que diez a medio hacer. Para un reclutador, el README es tu carta de presentación técnica.

30.1. Qué debe tener un buen README

- **Descripción clara** de qué es el proyecto y para qué sirve.
- **Diagrama de arquitectura** (aunque sea en texto): muestra que entiendes el sistema.
- **Stack técnico**: las tecnologías usadas, en una tabla.
- **Instalación**: los pasos, con el porqué de los comandos.
- **Problemas y soluciones**: la sección más valiosa. Demuestra capacidad real de resolver problemas.
- **Capturas de pantalla**: la prueba de que funciona de verdad.

¿Por qué la sección de problemas es la más valiosa? Porque cualquiera puede seguir un tutorial.

Lo que distingue a un buen perfil técnico es saber qué hacer cuando algo falla: leer el error, entender la causa, encontrar la solución. Documentar los problemas que resolviste —con causa, solución y aprendizaje— demuestra exactamente esa habilidad, que es la que de verdad se busca.

30.2. Añadir capturas de pantalla

Las imágenes del README son **capturas de tu pantalla** (no la imagen de Docker, que es otra cosa). Las haces con `Win` + `Shift` + `S`, las guardas como `.png`, las subes al repo y las enlazas con la sintaxis:

```
![Texto descriptivo](ruta/imagen.png)
```

Cuidado: antes de subir cualquier captura, comprueba que no se vea el token (el `#token=...` de la barra del navegador) ni claves API. El repositorio es público.

30.3. Qué subir y qué no

Subir (seguro)	NO subir (secretos / personal)
README.md	openclaw.json (tiene el token)
.gitignore	USER.md y MEMORY.md (datos personales)
SOUL.md, IDENTITY.md, AGENTS.md, TOOLS.md	.env y credenciales
Capturas sin token visible	La carpeta .openclaw completa

Consejo profesional: para los archivos con datos personales (USER.md, MEMORY.md), sube versiones "de ejemplo" (`USER.example.md`) con datos genéricos. Demuestra que conoces la buena práctica de no exponer datos reales, y queda mejor ante un reclutador.

PARTE IX

Práctica: Linux y uso real del asistente

Contenido aplicado: una guía de comandos de Linux para quien empieza, y cómo exprimir el asistente como herramienta de estudio con ejemplos concretos.

31. Linux desde cero: comandos para sobrevivir

Si estudias Linux (por ejemplo, con un libro como "The Linux Command Line" sobre una VM de Debian), esta guía te da los comandos esenciales explicados con criterio. Sirve como referencia mientras practicas, y tu asistente puede ampliarte cualquiera de ellos.

31.1. Orientarse en el sistema

Comando	Qué hace	Ejemplo
<code>pwd</code>	Muestra la carpeta actual.	<code>pwd</code>
<code>ls</code>	Lista archivos.	<code>ls -l</code> (con detalles)
<code>ls -la</code>	Lista todo, incluso archivos ocultos.	<code>ls -la</code>
<code>cd</code>	Cambia de carpeta.	<code>cd /etc</code>
<code>tree</code>	Muestra el árbol de carpetas.	<code>tree</code>

Archivos ocultos: en Linux, los archivos que empiezan por punto (como `.openclaw` o `.bashrc`) están "ocultos": no aparecen con `ls` normal, sino con `ls -a`. No es que estén protegidos; es solo una convención para no llenar la vista de archivos de configuración.

31.2. Ver y manipular archivos

Comando	Qué hace
<code>cat archivo</code>	Muestra todo el contenido de un archivo de texto.
<code>less archivo</code>	Muestra el archivo página a página (q para salir). Útil para archivos largos.
<code>head archivo</code>	Muestra las primeras líneas.
<code>tail archivo</code>	Muestra las últimas líneas. Con <code>-f</code> sigue en vivo (útil para logs).
<code>nano archivo</code>	Abre un editor de texto sencillo en la terminal.
<code>touch archivo</code>	Crea un archivo vacío (o actualiza su fecha).

31.3. Crear, copiar, mover, borrar

Comando	Qué hace
<code>mkdir carpeta</code>	Crea una carpeta.
<code>mkdir -p a/b/c</code>	Crea toda una ruta de carpetas anidadas de golpe.
<code>cp origen destino</code>	Copia un archivo.
<code>cp -r origen destino</code>	Copia una carpeta entera (recursivo).
<code>mv origen destino</code>	Mueve o renombra.
<code>rm archivo</code>	Borra un archivo (¡definitivo!).
<code>rm -r carpeta</code>	Borra una carpeta y todo su contenido.

Cuidado supremo con `rm -r`: borra carpetas enteras sin preguntar y sin papelera. Nunca lo ejecutes sobre una ruta de la que no estés absolutamente seguro. El comando `rm -rf /` (borrar todo desde la raíz) es el ejemplo clásico de orden catastrófica. Lee siempre dos veces antes de pulsar Enter.

31.4. Permisos: el modelo de seguridad de Linux

En Linux, cada archivo tiene permisos que controlan quién puede leerlo, escribirlo o ejecutarlo. Es uno de los conceptos que más cuesta al principio, así que vale la pena explicarlo.

Los permisos se dividen en tres grupos: el **propietario**, el **grupo** y **otros** (todos los demás). Y hay tres tipos de permiso: **leer (r)**, **escribir (w)** y **ejecutar (x)**.

El comando `chmod` cambia los permisos. Cuando ves algo como `chmod 755 archivo.sh`, ese número se descompone así:

Dígito	Para quién	755 significa
1º (7)	Propietario	leer + escribir + ejecutar (4+2+1)
2º (5)	Grupo	leer + ejecutar (4+1)
3º (5)	Otros	leer + ejecutar (4+1)

Cada permiso tiene un valor: leer = 4, escribir = 2, ejecutar = 1. Se suman. Así, 7 = 4+2+1 (todos), 5 = 4+1 (leer y ejecutar), 6 = 4+2 (leer y escribir).

Consejo: este es justo el tipo de tema donde tu asistente brilla. Pídele "explícame qué hace `chmod 644` desglosando cada número" y te lo detallará. Practicar con él acelera mucho el aprendizaje.

31.5. Gestión de paquetes en Debian/Ubuntu

En Debian (y derivados como Ubuntu), los programas se instalan con `apt`:

Comando	Qué hace
<code>sudo apt update</code>	Actualiza la lista de paquetes disponibles.
<code>sudo apt upgrade</code>	Actualiza los programas instalados.
<code>sudo apt install programa</code>	Instala un programa.
<code>sudo apt remove programa</code>	Desinstala un programa.

Sobre `sudo`: significa "ejecutar como superusuario" (administrador). Muchas tareas de sistema requieren privilegios elevados, y `sudo` los concede temporalmente para ese comando. Te pedirá tu contraseña. Es el equivalente a "ejecutar como administrador" en Windows.

31.6. Buscar cosas

Comando	Qué hace
<code>find . -name "*.txt"</code>	Busca archivos por nombre desde la carpeta actual.
<code>grep "texto" archivo</code>	Busca una palabra dentro de un archivo.
<code>grep -r "texto" .</code>	Busca en todos los archivos de una carpeta.

¿Por qué `grep` es tan importante? Porque buscar texto dentro de archivos es una de las tareas más frecuentes de un desarrollador: encontrar dónde se usa una función, localizar un error en un log, filtrar resultados. De hecho, ya lo usaste en este proyecto: `docker logs openclaw | findstr "listening"` en Windows es el equivalente de `grep` en Linux. El símbolo `|` (tubería o "pipe") encadena comandos: pasa la salida de uno como entrada del siguiente.

32. Usar el asistente para estudiar

Tener el asistente montado es solo la mitad. La otra mitad es saber pedirle las cosas para que de verdad te ayude a aprender. Aquí van estrategias y ejemplos concretos.

32.1. Configurar la personalidad de estudio

Lo primero es decirle cómo quieres que te trate. Un buen mensaje inicial:

```
Quiero que seas mi asistente de estudio. Recuerda siempre:
- Respóndeme en español.
- Explícame las cosas paso a paso, sin saltar detalles.
- Cuando me des código o comandos, explica qué hace cada parte.
- Si me equivoco, corrígeme y dime por qué.
- Asume que estoy aprendiendo: no des por sabido lo básico.
```

El agente guardará estas preferencias en su memoria y las aplicará en cada sesión.

32.2. Tipos de petición útiles

Para...	Pídele algo como...
Entender un concepto	"Explícame qué son los arrays en JavaScript con ejemplos sencillos."
Practicar	"Ponme 5 ejercicios de bucles for en PHP, de menos a más difícil."
Corregir	"Revisa este código y dime qué está mal y por qué."
Descifrar un comando	"¿Qué hace exactamente <code>chmod 755 script.sh</code> ?"
Resumir	"Resúmeme en 5 puntos los conceptos clave de los permisos en Linux."
Buscar info actual	"Busca en internet las novedades de Debian 13."

32.3. Buenas prácticas de "prompting"

La forma de pedir las cosas influye mucho en la calidad de la respuesta. Algunas reglas:

- **Sé específico.** "Explícame los bucles" es vago; "explícame la diferencia entre `for` y `while` en PHP con un ejemplo de cada uno" da mejor respuesta.
- **Da contexto.** Si estás con un ejercicio concreto, pégalo. El asistente trabaja mejor con información.
- **Pide el formato que quieres.** "En una tabla", "paso a paso", "en 3 frases". El asistente se adapta.
- **Itera.** Si la respuesta no te encaja, dilo: "no lo entiendo, explícamelo más simple" o "dame otro ejemplo".

32.4. Separar temas con sesiones

Cuando cambies de asignatura o de tema, usa `/new`. Eso mantiene cada conversación enfocada y evita que el asistente mezcle contextos. Tu personalidad y memoria se conservan; solo se limpia el hilo de la charla actual.

Consejo final: el asistente es una herramienta de aprendizaje, no un sustituto de pensar. Úsalo para que te explique, te corrija y te ponga ejercicios, pero intenta resolver tú primero. La combinación de esfuerzo propio más un tutor disponible a cualquier hora es enormemente eficaz.

33. Resumen del flujo completo

Para cerrar, aquí está todo el proceso condensado, de principio a fin, como mapa de referencia rápida.

33.1. Instalación (una sola vez)

1. Instalar Docker Desktop y comprobar que funciona.
2. Crear cuenta en build.nvidia.com y generar la clave API.
3. `docker pull` de la imagen de OpenClaw.
4. Definir la variable `NVIDIA_API_KEY`.
5. Ejecutar el onboarding y responder a las preguntas.
6. Arrancar el gateway con `docker run ... gateway`.
7. Ajustar el bind a `lan` y reiniciar.
8. Abrir el panel con la URL + token.
9. Hacer el bootstrap del agente (nombre, preferencias).
10. Activar la búsqueda web (DuckDuckGo) si la quieres.

33.2. Uso diario

1. Asegurarte de que el contenedor corre (`docker ps` ; si no, `docker start openclaw`).
2. Abrir el panel en el navegador con la URL guardada.
3. Chatear con el asistente. Usar `/new` al cambiar de tema.

33.3. Cuando algo falla

1. Mirar los logs: `docker logs --tail 40 openclaw`.
2. Pasar el diagnóstico: `docker exec openclaw openclaw doctor --fix`.
3. Validar la config: `docker exec openclaw openclaw config validate`.
4. Buscar el mensaje de error en el catálogo de problemas (capítulo 23).

Una última reflexión: montar este proyecto te ha enseñado, sin que quizá te dieras cuenta, un montón de cosas valiosas: Docker, redes, configuración de sistemas, APIs, lectura de logs, resolución de problemas y documentación. Eso es exactamente el tipo de aprendizaje práctico que más vale en desarrollo. No era trivial, y lo has hecho funcionar.

PARTE X

Fundamentos complementarios

Conceptos que aparecen por todo el manual y que merecen una explicación propia: redes, JSON, el panel de control y las dudas más frecuentes.

34. Redes desde cero: IP, puertos y localhost

A lo largo del manual aparecen términos de red —localhost, puerto 18789, bind, loopback— que conviene entender de verdad, porque son la causa de varios de los problemas típicos.

34.1. Qué es una dirección IP

Una **dirección IP** es el "número de teléfono" de un ordenador en una red. Identifica de forma única a cada dispositivo, para que los mensajes lleguen al sitio correcto. Tiene una forma como `192.168.1.45`. Cuando un programa quiere hablar con otro a través de la red, necesita conocer su IP.

34.2. localhost y 127.0.0.1

Hay una dirección IP especial: `127.0.0.1`, también llamada **localhost**. Significa "yo mismo", "esta misma máquina". Cuando abres `http://localhost:18789` en el navegador, le estás diciendo "conéctate a un servicio que corre en mi propio ordenador, en el puerto 18789".

¿Por qué "localhost" causa problemas en Docker? Aquí está la clave de uno de los errores más comunes. Para tu navegador en Windows, "localhost" es tu Windows. Pero **para un programa dentro de un contenedor, "localhost" es el propio contenedor**, no tu Windows. Son dos "yo mismo" distintos. Por eso, si el gateway escucha solo en su propio localhost (loopback), tu navegador de Windows no llega: están hablando de dos máquinas diferentes aunque parezca la misma. La solución —poner el bind en `lan`— hace que el gateway escuche en todas sus direcciones, no solo en su "yo mismo" interno.

34.3. Qué es un puerto

Un ordenador puede tener muchos servicios funcionando a la vez (un servidor web, una base de datos, OpenClaw...). Para distinguirlos, cada servicio "escucha" en un **puerto**: un número que actúa como una extensión telefónica. La IP te lleva al edificio (el ordenador); el puerto, a la oficina concreta (el servicio).

OpenClaw usa el puerto **18789**. Por eso accedes a él con `localhost:18789`: IP (localhost) + puerto (18789).

Puerto	Servicio habitual
80	Webs (HTTP)
443	Webs seguras (HTTPS)
22	SSH (acceso remoto a Linux)
18789	OpenClaw (panel de control)

34.4. Publicar un puerto en Docker

Como un contenedor está aislado también en red, su puerto interno no es accesible desde fuera salvo que lo "publiques". La opción `-p 18789:18789` conecta el puerto 18789 de tu ordenador con el 18789 del contenedor. El formato es `-p puerto_host:puerto_contenedor`.

Nota: los dos números pueden ser distintos. `-p 9000:18789` haría que accedieras desde `localhost:9000` aunque dentro el servicio use el 18789. En este proyecto se usan iguales por simplicidad.

34.5. bind: en qué direcciones escucha el gateway

El **bind** define en qué direcciones de red "se sienta a escuchar" un servicio:

Bind	Significa	Cuándo
<code>loopback</code>	Solo escucha en su propio "yo mismo" (127.0.0.1).	Instalación nativa, todo en la misma máquina.
<code>lan</code>	Escucha en todas sus interfaces de red.	Docker, donde el navegador llega "desde fuera" del contenedor.

35. JSON desde cero: para no romper la configuración

El archivo `openclaw.json` está en formato JSON. Cuando hay que editarlo a mano, un pequeño error de sintaxis impide que arranque. Entender JSON evita esos sustos.

35.1. Qué es JSON

JSON (*JavaScript Object Notation*) es un formato de texto para guardar datos estructurados. Es legible para humanos y para máquinas. Se basa en **pares de clave y valor**: un nombre (la clave) y su contenido (el valor).

```
{
  "nombre": "MiAsistente",
  "idioma": "español",
  "activo": true
}
```

35.2. Las reglas que más se incumplen

Regla	Explicación
Llaves <code>{ }</code>	Agrupar un conjunto de pares clave-valor (un "objeto").
Comillas en las claves	Las claves (y los textos) van entre comillas dobles.
Dos puntos <code>:</code>	Separan la clave de su valor.
Comas <code>,</code>	Separan un par del siguiente. El último de un bloque NO lleva coma.
Anidamiento	Un valor puede ser otro objeto <code>{ }</code> , creando niveles.

El error nº1 al editar JSON: las comas. La causa más común de "config inválida" al editar a mano es una coma de más o de menos. Regla práctica: dentro de un bloque `{ }`, todos los elementos llevan coma al final **menos el último**. Si borras una línea que era la última, asegúrate de quitar también la coma de la línea anterior. Y si añades una línea, ponle coma a la anterior.

35.3. Anidamiento: rutas de configuración

Cuando la documentación dice "pon `tools.web.search.enabled` en true", se refiere a navegar por los niveles anidados del JSON:

```
{
  "tools": {
    "web": {
      "search": {
        "enabled": true
      }
    }
  }
}
```

Esa ruta `tools.web.search.enabled` es el "camino" hasta el valor, separando cada nivel con un punto. Los comandos `config set` usan esa misma notación de puntos para llegar al valor sin que tengas que editar el archivo.

35.4. JSON5: la variante de OpenClaw

OpenClaw usa una variante llamada **JSON5**, más permisiva: admite comentarios (con `//`) y comas finales. Aun así, conviene seguir las reglas estrictas para evitar problemas.

Consejo: tras editar el JSON a mano, valida siempre antes de reiniciar: `docker exec openclaw openclaw config validate`. Si hay un error de sintaxis, te lo dirá, y podrás corregirlo antes de que rompa el arranque.

36. El panel de control por dentro

El panel web (Control UI) es desde donde chateas y gestionas el asistente. Conocer sus zonas te ayuda a moverte con soltura.

36.1. Zonas principales

Zona	Para qué sirve
Área de chat (centro)	Donde escribes y lees las respuestas del agente.
Selector de modelo (abajo)	Muestra el modelo activo (ej. "GLM 5.1 · Off") y permite cambiarlo.
Indicador de contexto	Muestra cuánta ventana de contexto llevas usada ("X / Y").
Caja de mensaje	Donde escribes. Enter para enviar.
Menú lateral	Sesiones, configuración del chat, ajustes.

36.2. El indicador "· Off" del modelo

Junto al nombre del modelo verás algo como "· Off". Eso indica el nivel de **"thinking"** (razonamiento extendido). Para los modelos de NVIDIA conviene dejarlo en **Off**, porque algunos solo admiten ese nivel; ponerlo en otro hace que la petición se rechace y el agente se quede colgado.

36.3. Botones de sugerencia

Al iniciar, el panel ofrece botones como "What can you do?" o "Help me configure a channel". Son atajos para explorar capacidades. Puedes ignorarlos y escribir directamente lo que necesites.

37. Preguntas frecuentes

37.1. Sobre el funcionamiento

¿Tengo que tener el PC encendido para que funcione? Sí. El gateway corre en tu equipo. Si lo apagas, el asistente deja de responder, incluso si le hablas desde el móvil por un canal.

¿Pierdo la configuración si borro el contenedor? No, siempre que hayas usado volúmenes (`-v`). Los datos viven en tu Windows, no en el contenedor. Puedes borrar y recrear el contenedor sin perder nada.

¿Puedo cambiar de modelo sin perder la personalidad? Sí. La personalidad y la memoria están en los archivos del workspace, que son independientes del modelo. Cambias el "cerebro" sin cambiar "quién es" el agente.

37.2. Sobre los costes

¿Cuánto cuesta esto? Con el tier gratuito de NVIDIA, nada. Docker es gratis, OpenClaw es software libre, y la API de NVIDIA tiene un nivel gratuito. Solo pagarías si decidieras usar un proveedor de modelos de pago.

¿Los tokens me cuestan dinero? Con NVIDIA gratuito, no. El contador de tokens importa por la memoria del modelo, no por la factura. Con proveedores de pago, sí.

37.3. Sobre los problemas

El asistente mezcla palabras de otros idiomas, ¿es un fallo? No es un fallo de configuración: es una tendencia de algunos modelos multilingües. Se reduce reforzando la regla de idioma en el `SOUL.md`, o cambiando a un modelo menos propenso a mezclar.

¿Por qué a veces el agente "se queda pensando" y no responde? Las causas más comunes son: un modelo demasiado lento que agota el tiempo de espera, un nivel de "thinking" no soportado, o un modelo que no maneja bien las herramientas. El catálogo de problemas (capítulo 23) detalla cada caso.

¿Es seguro? Para uso personal en tu equipo, accesible solo desde tu navegador con token, sí. La precaución importante es no exponer el panel a internet sin protección y no subir secretos a repositorios públicos.

37.4. Sobre el aprendizaje

¿Necesito saber programar para montar esto? No para montarlo siguiendo esta guía. Pero hacerlo te enseña conceptos muy valiosos (Docker, redes, configuración, resolución de problemas) que sí son la base del desarrollo. Es un excelente proyecto de aprendizaje.

¿Esto vale para un portfolio? Mucho. Demuestra que sabes desplegar software, integrar APIs, diagnosticar problemas y documentar. La Parte VIII explica cómo presentarlo en GitHub para que luzca ante un reclutador.

PARTE XI

Para profundizar

Tres temas para quien quiere ir más allá: entender cómo funciona la IA por dentro, la instalación alternativa con Node.js, y la automatización del asistente.

38. Cómo funciona la IA por dentro (sin matemáticas)

No necesitas entender los detalles técnicos para usar OpenClaw, pero una idea básica de cómo "piensa" un modelo de lenguaje te ayuda a usarlo mejor y a entender sus límites.

38.1. Predecir la siguiente palabra

En el fondo, un modelo de lenguaje hace una cosa aparentemente simple: **predecir qué palabra viene después**. Dado un texto, calcula cuál es la continuación más probable, palabra a palabra (en realidad, token a token). Cuando le escribes una pregunta, va generando la respuesta eligiendo cada vez el siguiente fragmento más plausible según todo lo que aprendió.

Lo sorprendente es que, de esa mecánica tan básica repetida a gran escala y entrenada con cantidades enormes de texto, emerge la capacidad de responder preguntas, escribir código, razonar y conversar. No hay una base de datos de respuestas: hay un sistema que ha aprendido los patrones del lenguaje y el conocimiento que contiene.

38.2. El entrenamiento

Un modelo se crea en una fase de **entrenamiento**: se le muestran cantidades gigantescas de texto (libros, webs, código...) y aprende los patrones. Este proceso es carísimo y lento, y lo hacen empresas con grandes recursos. Una vez entrenado, el modelo queda "congelado": ya no aprende más por sí mismo. Por eso tiene una "fecha de corte" de conocimiento, y por eso necesita herramientas como la búsqueda web para saber cosas recientes.

¿Por qué el asistente a veces se equivoca o "inventa"? Porque genera texto probable, no verdad verificada. Si no sabe algo, a veces produce una respuesta que "suena bien" pero es incorrecta (lo que se llama "alucinación"). Por eso conviene verificar datos importantes, y por eso la búsqueda web es tan útil: le da información real y actual en la que basarse en vez de depender solo de lo que memorizó.

38.3. Por qué la memoria es externa

Como el modelo está "congelado" tras el entrenamiento, no puede recordar tus conversaciones por sí mismo. Ahí entra OpenClaw: guarda la memoria en archivos (el workspace) y se la "recuerda" al modelo al inicio de cada sesión. El modelo no cambia; lo que cambia es la información que OpenClaw le proporciona cada vez. Es como un experto con amnesia al que le pasas tus notas antes de cada reunión.

38.4. Qué es el "contexto" en este marco

Con esto se entiende mejor la ventana de contexto: es todo el texto que el modelo "ve" de una vez para generar su respuesta —tu mensaje, el historial, tus notas del workspace—. Cuanto más grande, más puede tener en cuenta. Pero es limitada, y por eso, cuando se llena, OpenClaw resume lo viejo para hacer sitio a lo nuevo.

39. Comparativa de rendimiento y recursos entre métodos

Ya viste cómo instalar con cada método (Parte IV). Aquí va una comparación práctica de qué esperar de cada uno en el día a día, para decidir con criterio si tienes dudas.

Aspecto	Node nativo	Docker
Arranque inicial	Inmediato tras instalar Node.	Requiere descargar o construir la imagen primero.
Uso de RAM/disco en reposo	Ligeramente menor (sin capa de contenedor).	Algo más, por la capa de aislamiento, normalmente poco significativo.
Limpieza tras desinstalar	Quedan dependencias de Node en el sistema.	Borrar el contenedor no deja rastro.
Actualizar OpenClaw	<code>npm update -g openclaw</code> o reinstalar.	<code>docker pull</code> de una versión nueva y recrear el contenedor.
Aislamiento de fallos	Un fallo grave puede afectar a otras cosas instaladas con Node.	Un fallo queda contenido dentro del contenedor.

En la práctica, para la mayoría de usos personales, la diferencia de rendimiento entre ambos métodos es mínima — la elección suele decidirse más por preferencia de mantenimiento (¿prefieres gestionar versiones de Node, o gestionar contenedores?) que por velocidad real. Si dudas, Docker tiende a ser la opción más fácil de "deshacer" sin dejar rastro si decides probar y no convencerte.

40. Automatización: hooks y tareas programadas

OpenClaw puede hacer cosas por su cuenta, sin que se lo pidas en cada momento: guardar memoria automáticamente, comprobar cosas cada cierto tiempo, ejecutar tareas a una hora fija. Es un tema avanzado, pero conviene saber que existe.

40.1. Hooks: acciones disparadas por eventos

Un **hook** es una acción que se ejecuta automáticamente cuando ocurre un evento. El ejemplo más útil es el de *memoria de sesión*: cuando escribes `/new`, un hook puede guardar automáticamente lo importante de la conversación en la memoria antes de empezar la sesión nueva. Así no pierdes lo aprendido al cambiar de sesión.

40.2. Heartbeats: comprobaciones periódicas

Un **heartbeat** ("latido") es una señal periódica que permite al agente hacer comprobaciones cada cierto tiempo —revisar el correo, el calendario, etc.— y avisarte si hay algo relevante. Se configura con moderación, porque cada comprobación consume recursos.

40.3. Cron: tareas a hora fija

Cron es un sistema clásico de Linux para programar tareas a horas exactas ("todos los lunes a las 9:00"). OpenClaw lo integra para tareas que requieren precisión horaria, como recordatorios.

Mecanismo	Cuándo se usa
Hook	Reacciona a un evento (ej. guardar memoria al hacer <code>/new</code>).
Heartbeat	Comprobaciones periódicas flexibles (cada ~30 min).
Cron	Tareas a una hora exacta y repetitiva.

Para empezar, no necesitas nada de esto. La automatización es para cuando ya dominas lo básico y quieres que el asistente trabaje de forma proactiva. Un asistente de estudio funciona perfectamente sin hooks ni cron. Está aquí para que sepas que la posibilidad existe y puedas explorarla cuando te apetezca.

40.4. Una visión de futuro

Estas capacidades son lo que diferencia a un asistente conversacional de un verdadero "agente": la posibilidad de actuar por iniciativa propia dentro de los límites que tú definas. A medida que cojas soltura, podrás convertir tu agente en algo que no solo responde cuando le hablas, sino que te avisa, te recuerda y te ayuda de forma proactiva. Pero todo a su tiempo: primero lo sólido, luego lo vistoso.

PARTE XII

La vista de administración del panel

Una exploración completa y verificada de la consola de administración de OpenClaw: Ajustes, Control y Agente, sección por sección, con datos reales.

43. Mapa general del panel de administración

Más allá del chat, OpenClaw incluye una consola de administración completa. Se organiza en cuatro grupos en el menú lateral, y dentro de "Ajustes" hay además ocho grandes pestañas. Este capítulo da el mapa; los siguientes entran en cada pieza con detalle real, verificado sección por sección sobre una instalación en marcha.

43.1. Los cuatro grupos del menú lateral

Grupo	Contiene
CHAT	La conversación con el agente.
CONTROL	Resumen, Actividad, Panel de trabajo, Instancias, Sesiones, Uso, Tareas Cron — monitorización y operación del gateway.
AGENTE	Agentes, Skills, Taller de Skills, Nodos, Sueños — identidad y capacidades del agente.
AJUSTES	La configuración completa, organizada en pestañas.

43.2. Las pestañas de Ajustes

Pestaña	Qué agrupa
Ajustes (Quick Settings)	Modelo, seguridad, perfil personal, apariencia, automatizaciones — un resumen de acceso rápido.
Canales	Configuración detallada de cada canal de mensajería soportado.
Comunicaciones	Mensajes, Broadcast, Notifications, Talk (voz), Audio — comportamiento transversal de mensajería.
Apariencia	Theme, UI, Setup Wizard.
Automatización	Commands, Hooks, Bindings, Cron, Approvals, Plugins.
MCP	Servidores de Model Context Protocol.
Infraestructura	Gateway, Web, Browser, Node Host, Discovery, Media, ACP, MCP.
IA y agentes	Agents, Models, Skills, Tools, Memory, Session.
Depuración	Snapshots de estado/salud, RPC manual, catálogo de modelos.
Registros	El log de archivo del gateway, en vivo, con filtros.

Algunas secciones aparecen en más de un sitio del menú (por ejemplo, MCP tiene pestaña propia y también aparece dentro de Infraestructura) — es el mismo ajuste accesible por dos caminos de navegación distintos, no información duplicada.

44. Ajustes: Comunicaciones

Todo lo relativo a cómo entran y salen los mensajes: por canal, en general, y por tipo de medio (texto, voz, audio).

44.1. Channels: el catálogo completo de canales

OpenClaw soporta muchos más canales de los habituales. El catálogo real incluye más de 20 opciones:

Canal	Nota
Telegram	El más sencillo de arrancar: bot vía @BotFather.
Discord	"Very well supported right now" según la propia interfaz.
WhatsApp	Recomienda número y eSIM separados de tu personal.
Slack	Vía Socket Mode.
Signal	Configuración más laboriosa (signal-cli).
iMessage	Requiere bridge en macOS.
SMS	Vía Twilio.
Matrix, Mattermost, Tlon	Requieren instalar un plugin aparte para activarse.
Microsoft Teams	Soporte empresarial, incluye entornos gubernamentales (USGov/ China).
Feishu/Lark, LINE, Zalo, QQ Bot	Plataformas regionales (China, Japón/Corea, Vietnam).
IRC, Nostr	Redes clásicas/descentralizadas.
Google Chat, Nextcloud Talk, Synology Chat	Integraciones de plataformas concretas.

El patrón de seguridad común a todos: cada canal tiene un campo `DM Policy` con cuatro opciones idénticas: `pairing` (recomendada), `allowlist`, `open`, `disabled`. Aprender este patrón una vez te sirve para entender la seguridad de los 23 canales a la vez.

44.2. Messages: reglas generales de mensajería

Configuración transversal, independiente del canal concreto:

- **Colas de mensajes entrantes:** si llegan varios mensajes mientras el agente ya responde, cuatro modos posibles — `steer` (inyecta en la respuesta activa), `followup` (lo deja para después), `collect` (agrupa varios), `interrupt` (corta la respuesta en curso).

- **Reacciones de estado:** el emoji va cambiando según el progreso (en cola → pensando → usando herramienta → terminado/error).
- **Patrones de mención en grupo:** expresiones que detectan cuándo el bot debe responder en un chat grupal, para no contestar a todo.
- **Texto-a-voz (TTS):** política para leer las respuestas en voz alta en superficies que lo soporten.

44.3. Broadcast: enviar a varios destinos

Una sola opción relevante: `Broadcast Strategy`, con dos modos — `parallel` (envía a todos los destinos a la vez, más rápido) o `sequential` (uno detrás de otro, con control estricto de orden y sin saturar de golpe).

44.4. Notifications: avisos del navegador

Controla si el panel puede mandarte notificaciones push del sistema operativo (como las de WhatsApp) cuando el gateway tiene algo que avisarte, sin que tengas la pestaña abierta mirando. Requiere conceder permiso explícito al navegador.

44.5. Talk: conversación de voz en tiempo real

OpenClaw soporta voz real, no solo texto-a-voz simple. Tres modos de ejecución:

Modo	Qué hace
<code>realtime</code>	Conversación de voz fluida en vivo, como una llamada.
<code>stt-tts</code>	Transcribe a texto → el agente responde en texto → se convierte de vuelta a voz.
<code>transcription</code>	Solo transcribe, sin respuesta hablada.

El "cerebro" detrás de la voz (`Talk Realtime Brain: agent-consult`) consulta al **mismo agente completo**, con su memoria y personalidad — hablar por voz con el agente no sería "otro asistente", sigue siendo él, solo que por un canal de audio.

También se puede ajustar si el agente se interrumpe cuando empiezas a hablar (activado por defecto, para una conversación natural), el silencio necesario para considerar que terminaste de hablar (~700-900ms según plataforma), y el idioma de reconocimiento de voz.

44.6. Audio: transcripción de archivos

Distinto de Talk: esto procesa **archivos de audio** (notas de voz recibidas), no conversación en vivo. Usa un comando externo configurable (por ejemplo, `whisper-cli`) — el mismo patrón de "motor desacoplado" que vimos con la búsqueda web. En una instalación nueva, este comando está vacío: la transcripción de audio no funciona hasta que se configura explícitamente.

45. Ajustes: Apariencia

45.1. Theme

Cuatro familias de tema (Claw, Knot, Dash, e Import para traer un tema externo de la herramienta "tweakcn"), control de redondez de bordes y tamaño de texto. La pantalla también muestra el estado de conexión en vivo: `ws://localhost:18789` — confirma que el panel habla con el gateway por **WebSocket**, no HTTP normal, porque un chat necesita una conexión permanente y bidireccional (el servidor "empuja" la respuesta palabra a palabra, en vez de que el navegador esté preguntando constantemente si ya hay novedades).

45.2. UI: identidad visual del asistente

Distinto del archivo `IDENTITY.md` real: este es un **override puramente visual** del panel — nombre, avatar y color de acento mostrados en la interfaz, que puede no coincidir con lo que el agente "sabe" de sí mismo si se borra este override (vuelve a leerse de `IDENTITY.md`).

45.3. Setup Wizard

Un registro de auditoría de solo lectura: cuándo se ejecutó por última vez el onboarding, con qué comando, con qué versión de OpenClaw y en qué modo (local/remoto). Útil para diagnóstico, no para editar.

46. Ajustes: Automatización

46.1. Commands: comandos de chat

Interruptores —todos en `Off` por defecto— para comandos potentes ejecutables desde el propio chat: `/bash` (shell del host), `/config` (leer/escribir config), `/debug`, `/mcp`, `/plugins`. También `Command Owners`: una lista explícita de quién puede ejecutar comandos con privilegios de propietario — vacía por defecto, que es justo lo que el primer `openclaw doctor` de este manual avisaba sin configurar.

46.2. Hooks: eventos entrantes

Permite que **servicios externos llamen al gateway por HTTP** para despertar al agente, en vez de que solo reaccione a mensajes de chat. Desactivado por defecto, protegido con un token de confianza total. Incluye un sub-sistema completo para **Gmail**: notificaciones push de correo nuevo vía Google Pub/Sub, con filtro por etiqueta y control de qué contenido se procesa.

46.3. Bindings: enrutamiento entre agentes

Las reglas que deciden a qué agente va cada mensaje (`type=route`) — la versión visual de lo que vimos en la Parte VII (multi-agente). También existe `type=acp`, para vínculos persistentes con runtimes de agentes externos (ver el capítulo de ACP más adelante).

46.4. Cron: motor de tareas programadas

El sistema completo detrás de "Tareas Cron" del menú lateral: concurrencia máxima (8 tareas a la vez por defecto), reintentos automáticos con espera creciente ante fallos transitorios (30s → 60s → 5min), alertas configurables si una tarea falla, e historial de ejecuciones guardado en SQLite (2000 filas por tarea por defecto).

46.5. Approvals: reenvío de aprobaciones

Permite que las peticiones de aprobación (ejecutar un comando, usar un plugin) se reenvíen a un canal distinto del de origen — pensado para equipos con un operador de guardia, no para uso personal con un único usuario.

46.6. Plugins: el catálogo completo

Más de 30 proveedores de modelos de IA (Anthropic, OpenAI, Google, NVIDIA, Mistral, Ollama, llama.cpp, LM Studio, xAI, y muchos más), cada canal de mensajería como plugin independiente, y utilidades como extracción de documentos, lectura web, diagnóstico (OpenTelemetry/Prometheus) o

control de voz/teléfono. Permite allowlist/denylist de plugins y "slots exclusivos" (solo un plugin puede ser el proveedor de memoria o de motor de contexto a la vez).

47. Ajustes: MCP y ACP — extensibilidad avanzada

47.1. MCP: conectar herramientas externas

MCP (Model Context Protocol) es un estándar abierto para conectar el agente con herramientas y fuentes de datos externas de forma uniforme, sin tener que programar cada integración a medida. En una instalación nueva no hay servidores configurados.

47.2. ACP: conectar con otros agentes

ACP (Agent Communication Protocol) resuelve algo distinto: conectar con **otros runtimes de IA completos**, no con herramientas. Desactivado por defecto, aunque la pieza de soporte (`acpx`) ya viene instalada como plugin y skill.

La diferencia en una frase: **MCP conecta con herramientas; ACP conecta con otros "cerebros"**.

48. Ajustes: Infraestructura

48.1. Gateway: el servidor en detalle

Cinco modos de bind (no solo loopback/lan): `auto`, `lan`, `loopback`, `custom`, `tailnet`. El token se muestra oculto por defecto. Tres vías de acceso remoto: Tailscale, TLS directo, o un gateway remoto por SSH. Un hallazgo de seguridad notable: hay una **lista de comandos de nodo bloqueados por defecto** aunque el dispositivo lo permitiera — `camera.snap`, `camera.clip`, `screen.record`, `contacts.add`, `calendar.add`, `reminders.add`, `sms.send`, `sms.search` — una defensa de fábrica contra usos invasivos de la privacidad.

48.2. Web: fiabilidad de conexión

Política de reconexión con backoff exponencial y "jitter" (aleatoriedad) para evitar que todos los clientes reconecten a la vez tras una caída ("estampida"). Temporizadores específicos para WhatsApp Web (que usa una librería, Baileys, que simula una sesión de navegador, al no existir API de bot oficial gratuita para WhatsApp).

48.3. Browser: navegación autónoma

El agente puede controlar un Chrome real (local o remoto) vía CDP (Chrome DevTools Protocol), no solo leer páginas. Incluye una protección importante contra **SSRF** (Server-Side Request Forgery): bloquea por defecto que el navegador del agente acceda a rangos de red privados, para que un sitio malicioso no pueda usarlo para "rebotar" peticiones hacia tu red local.

48.4. Node Host, Discovery y Media

Node Host expone capacidades de esta máquina (como el navegador) a otros nodos remotos. **Discovery** controla cómo el gateway se anuncia en la red local (mDNS) o a larga distancia. **Media** gestiona archivos adjuntos: por defecto les pone un nombre temporal generado (no el original) y, salvo que se configure un TTL, nunca se borran automáticamente.

49. Ajustes: IA y Agentes

49.1. Agents: los valores por defecto del workspace

Aquí vive, en formato visual, todo lo que editamos a mano en el `openclaw.json`. Datos clave: el bootstrap de cada archivo del workspace se trunca a 20.000 caracteres por archivo y 60.000 en total por defecto — protección directa contra que `MEMORY.md` crezca sin control y devore la ventana de contexto. También límites específicos para PDF: **10 MB y 20 páginas máximo** — un dato a tener en cuenta si un archivo adjunto falla de forma extraña.

49.2. Models: el catálogo real y la clave del timeout

A modo de ejemplo, esta es la ventana de contexto de algunos modelos típicos servidos por un proveedor en la nube:

Modelo	Ventana de contexto	Tokens de salida máx.
Nemotron 3 Ultra 550B	1.000.000	16.384
Nemotron 3 Super 120B	262.144	8.192
Kimi K2.5	262.144	8.192
GLM 5.1 / GLM-5	202.752	8.192
MiniMax M2.5 / M2.7	196.608	8.192

El dato más importante de todo este capítulo: la clave real para subir el tiempo de espera ante un modelo lento **no** es una clave global del agente — es `models.providers.<proveedor>.requestTimeout`, a nivel de cada proveedor. Tiene sentido: la paciencia necesaria depende del servidor concreto que responde, no del agente que pregunta. Si un modelo concreto te resulta lento, este es el ajuste correcto — no el que se intentó sin éxito en el capítulo de troubleshooting.

49.3. Tools: el control más fino sobre lo que puede hacer el agente

Cuatro perfiles de herramientas: `minimal`, `coding`, `messaging`, `full`. Y un hallazgo de seguridad relevante: **Tool-loop Detection** **está desactivado por defecto** — el sistema que detecta y corta automáticamente una cadena de llamadas a herramientas repetida sin avanzar. Activarlo (con sus umbrales de aviso a 10, crítico a 20, y un "disyuntor" global a 30 intentos sin progreso) es la defensa directa contra ese tipo de bucle.

49.4. Memory: más allá de MEMORY.md — el sistema QMD

El sistema básico (`builtin`) que usa este manual carga archivos enteros al prompt. Existe un backend alternativo, **QMD**, con búsqueda semántica real: indexa la memoria con embeddings y recupera solo lo relevante para cada consulta, en vez de cargarlo todo. Incluye incluso indexación experimental de transcripciones completas de sesiones pasadas (desactivada por defecto, por ruido y coste de almacenamiento).

49.5. Session: por qué la limpieza de sesiones no encontraba nada que borrar

Resuelve un cabo suelto real de este manual: el comando `sessions cleanup --dry-run` decía "remove 0" porque **ninguna política de retención estaba configurada** — sin un `Session Prune After` (edad) o `Session Max Entries` (cantidad) definidos, no había ningún criterio de "esto es viejo" que aplicar. También existe reset automático de sesiones por horario (`daily`) o por inactividad (`idle`) — un `/new` programado, sin que el usuario tenga que escribirlo.

50. Depuración y Registros

50.1. Depuración: el estado interno en crudo

Tres bloques de datos JSON puros: **Status** (versión, heartbeat, sesiones, salud del bucle de eventos de Node.js), **Health** (si todo está bien, qué plugins cargaron de verdad — en una instalación típica: `acpx, browser, canvas, device-pair, duckduckgo, file-transfer, memory-core, phone-control, talk-voice`), y un **RPC manual** para enviar cualquier método crudo del gateway directamente.

50.2. Registros: la traza exacta de un incidente

El mismo archivo que se vería con `docker logs`, en una vista filtrable del panel. Permite reconstruir, minuto a minuto, qué intentó el agente cuando algo salió mal: por ejemplo, una secuencia de intentos fallidos de `config.patch` con distintos errores de formato sucesivos (`raw required` → error de sintaxis JSON5 → `raw must be an object`) revela que el agente estaba probando claves de configuración por ensayo y error, sin dar con el formato exacto que la API exige — y, frecuentemente, sin haber dado tampoco con la *ruta* correcta del ajuste.

Cuando algo falle de forma confusa en el chat, "Registros" (o `docker logs --tail 40 openclaw` desde la terminal) es siempre el primer sitio donde mirar: ahí está la secuencia exacta de qué se intentó y por qué falló cada paso.

51. Grupo Control: monitorización del gateway

51.1. Resumen

El panel de un vistazo: estado de conexión, coste acumulado (cero si usas un proveedor con tier gratuito), número de sesiones, skills activas, tareas cron. Incluye un aviso útil de "skills con dependencias faltantes" — muchas skills oficiales (1Password, Apple Notes/Reminders...) requieren binarios o cuentas que no existen en un entorno Windows/Docker, y aparecen marcadas sin que eso sea un error real.

51.2. Actividad

No es el log general del sistema (eso es "Registros") — es específicamente el seguimiento de **herramientas con ejecución visual del lado del navegador** (como el navegador automatizado o Canvas), con filtros por estado (en ejecución/completado/error).

51.3. Panel de trabajo (Workboard)

Un tablero Kanban completo para el agente, desactivado por defecto (plugin `workboard`). Columnas: Triage, Pendientes, Por hacer, Programado, Listo, En ejecución, Revisión, Bloqueado, Completado. Pensado para tareas estructuradas que se planifican y revisan, no solo conversación lineal. Se activa con:

```
docker exec openclaw openclaw config set plugins.entries.workboard.enabled true
docker restart openclaw
```

51.4. Instancias

Quién está conectado al gateway ahora mismo. En una instalación Docker típica aparecen dos entradas: el propio contenedor (con su IP interna en el rango privado de la red de Docker, típicamente `172.17.0.x` — y, si instalaste sobre Windows, la firma del kernel `-microsoft-standard-WSL2` confirma que corre sobre el kernel de WSL2) y el navegador conectado desde el host (la otra dirección del mismo rango, la puerta de enlace de la red Docker — es decir, "el host visto desde dentro del contenedor").

Esta es la confirmación de campo de por qué el bind `loopback` falla en Docker: el contenedor (`.2`) y el host (`.1`) son direcciones distintas dentro de la red virtual de Docker. "Loopback" en el contenedor nunca verá una conexión que llega desde `.1`.

51.5. Sesiones (vista visual)

Una tabla de sesiones activas con búsqueda libre, mostrando para cada una su agente, tipo de chat, estado, y si sus ajustes (modelo, thinking) están fijados o **heredados** de los valores por defecto del agente — el mismo patrón de herencia que se ve en la configuración técnica, aplicado a nivel de sesión individual.

51.6. Uso: el balance cuantificado

El panel de analítica más completo: mensajes totales, coste, llamadas a herramientas por tipo, tasa de error, ranking de modelos/proveedores/canales usados, y desglose por sesión individual con su duración y número de errores. Permite identificar, con datos objetivos, en qué sesión concreta se concentró un incidente — por ejemplo, una sesión corta con un número desproporcionado de llamadas a la herramienta `gateway` suele señalar una cadena de intentos fallidos de cambiar configuración.

51.7. Tareas Cron (vista visual)

Creación de tareas programadas **en lenguaje natural**, sin necesidad de escribir una expresión cron tradicional — los campos avanzados quedan disponibles pero no son obligatorios. Incluye historial de ejecuciones con su estado de entrega.

52. Grupo Agente: identidad y capacidades

52.1. Agentes

Vista del agente `main` con pestañas: Overview, Archivos (los 7 del workspace, editables directamente con vista previa de Markdown), Herramientas, Skills, Channels, Cron Jobs — todo centralizado por agente.

52.2. Skills (gestión)

De las skills instaladas en una configuración típica (decenas, según el catálogo del momento), normalmente solo una fracción está realmente lista para usar sin configuración adicional; el resto necesita claves de API o binarios que no están presentes en un entorno Windows/Docker. Esto no es un fallo: es el catálogo completo de capacidades disponibles si se configuran, frente a las que ya funcionan "de fábrica".

52.3. Taller de Skills

Donde aparecerían propuestas de skills nuevas generadas por el propio agente, si el modo autónomo estuviera activado. Vacío por defecto.

52.4. Nodos (gestión)

Gestión de dispositivos emparejados, con sus roles y permisos (*scopes*) exactos. Aquí puede reaparecer el mismo error de permisos `EPERM: chmod '/home/node/.openclaw'` visto en los logs desde el primer `doctor` de la instalación — un problema de propiedad de archivos entre Windows y el contenedor, de bajo impacto pero persistente, que bloquea específicamente la carga de aprobaciones de ejecución desde esta pantalla.

52.5. Sueños

La sección más singular del panel: una escena visual nocturna (cielo estrellado, luna, el avatar del agente dormido) que representa la consolidación de memoria en segundo plano, modelada explícitamente sobre las fases reales del sueño humano: **ligero, profundo y REM** — las mismas fases donde la ciencia del sueño sitúa la consolidación de memoria en un cerebro humano. Tres pestañas: Escena (la animación), Diario (el registro textual de lo consolidado) y Avanzado (configuración). Inactivo por defecto.

Con esto se completa la exploración de los tres grupos del panel de administración (Ajustes, Control, Agente), verificados sobre una instalación real en marcha. El panel de chat, que es donde se vive el día a día, queda documentado en las primeras partes de este manual.

Apéndice A. Chuleta de comandos

A.1. Terminal (PowerShell / Bash)

Acción	PowerShell	Bash
Dónde estoy	<code>pwd</code>	<code>pwd</code>
Listar archivos	<code>ls</code>	<code>ls</code>
Entrar en carpeta	<code>cd carpeta</code>	<code>cd carpeta</code>
Variable de entorno	<code>\$env:CLAVE="x"</code>	<code>export CLAVE="x"</code>
Abrir archivo	<code>notepad archivo</code>	<code>nano archivo</code>
Continuar línea	acento grave <code>`</code>	barra <code>\</code>

A.2. Docker

Acción	Comando
Descargar imagen	<code>docker pull imagen</code>
Crear y arrancar contenedor	<code>docker run -d --name X imagen</code>
Ver corriendo	<code>docker ps</code>
Arrancar / parar / reiniciar	<code>docker start/stop/restart X</code>
Ver logs	<code>docker logs -f X</code>
Ejecutar comando dentro	<code>docker exec X comando</code>
Entrar al contenedor	<code>docker exec -it X bash</code>
Borrar contenedor	<code>docker rm -f X</code>

A.3. OpenClaw — comando nativo vs Docker

Acción	Node nativo	Docker (anteponer)
Configuración inicial	<code>openclaw onboard</code>	<code>docker exec -it openclaw openclaw onboard</code>
Leer config	<code>openclaw config get <clave></code>	<code>docker exec openclaw openclaw config get <clave></code>
Cambiar config	<code>openclaw config set <clave> <valor></code>	<code>docker exec openclaw openclaw config set <clave> <valor></code>
Validar config	<code>openclaw config validate</code>	<code>docker exec openclaw openclaw config validate</code>
Diagnóstico	<code>openclaw doctor --fix</code>	<code>docker exec openclaw openclaw doctor --fix</code>
URL del panel	<code>openclaw dashboard --no-open</code>	<code>docker exec openclaw openclaw dashboard --no-open</code>
Listar modelos	<code>openclaw models</code>	<code>docker exec openclaw openclaw models</code>
Crear agente	<code>openclaw agents add <nombre></code>	<code>docker exec -it openclaw openclaw agents add <nombre></code>
Arrancar / parar / reiniciar	<code>openclaw gateway start/stop/restart</code>	<code>docker start/stop/restart openclaw</code>

La regla general: en Docker, cualquier comando nativo de la columna izquierda se traduce anteponiendo `docker exec openclaw` (o `docker exec -it openclaw` si es interactivo).

A.4. Comandos dentro del chat

Comando	Efecto
<code>/new</code>	Sesión nueva (recarga workspace).
<code>/model</code> · <code>/models</code>	Cambia / lista modelos.
<code>/context list</code>	Contexto cargado.
<code>/help</code>	Ayuda.

Apéndice B. Glosario completo

Término	Significado
Agente	La IA con personalidad, workspace y memoria. Puede haber varios.
API	Interfaz que permite que dos programas se comuniquen. La de NVIDIA sirve los modelos.
Bash	Intérprete de comandos típico de Linux y de dentro de los contenedores.
Bind	La interfaz de red donde escucha el gateway (loopback = solo local; lan = todas).
Bootstrap	Ritual de primer arranque que fija la identidad del agente.
Canal	Vía de entrada: panel web, WhatsApp, Telegram, etc.
CLI	Interfaz de línea de comandos (escribir órdenes de texto).
Commit	Una "foto" guardada del estado de un proyecto en Git.
Compaction	Resumen automático del contexto cuando se llena la ventana.
Contenedor	Una imagen Docker en ejecución; entorno aislado y desechable.
Docker	Plataforma para ejecutar aplicaciones en contenedores aislados.
Gateway	El proceso central de OpenClaw; corre en tu equipo.
GitHub	Servicio web para alojar y compartir repositorios de Git.
GUI	Interfaz gráfica (ventanas, botones, ratón).
Imagen	Plantilla congelada con una app y su entorno, lista para ejecutar.
LLM	Modelo de lenguaje grande; la IA que genera las respuestas.
Loopback	La dirección "yo mismo" (127.0.0.1) de una máquina.
Modelo	El LLM concreto que usa el agente (Nemotron, GLM...).
NIM	Plataforma de inferencia de NVIDIA, compatible con OpenAI.
Onboarding	Asistente de configuración inicial de OpenClaw.
Prompt	El indicador de la terminal que espera tu comando.
Puerto	"Ventana" de red para acceder a un servicio (aquí, 18789).
Sesión	Conversación con historial; se reinicia con /new.
Shell	El intérprete que ejecuta los comandos (PowerShell, Bash...).
Token (modelo)	Unidad mínima de texto que maneja el modelo.
Token (gateway)	Clave de acceso al panel de control.
Tool calling	Capacidad del modelo de usar herramientas (como la búsqueda web).
Volumen	Conexión de una carpeta del host con el contenedor, para persistir datos.
Workspace	Carpeta con los archivos .md que definen al agente.
WSL2	Subsistema que permite ejecutar Linux dentro de Windows.

Apéndice C. Recursos y enlaces

Recurso	Para qué
docs.openclaw.ai	Documentación oficial de OpenClaw.
github.com/openclaw/openclaw	Código fuente del proyecto.
build.nvidia.com	Registro de cuenta y generación de la clave API de NVIDIA.
docker.com	Descarga de Docker Desktop y documentación.

Fin del Manual · Documento de referencia general, independiente del método de instalación o ubicación elegidos ·
Basado en la documentación oficial de OpenClaw (docs.openclaw.ai) · OpenClaw es software libre con licencia MIT.